

A Privacy-Constrained, Locally-Hosted Retrieval-Augmented Assistant for IT Incident Resolution

Ayoub Aarab
Internship Report
NTT DATA

August 28, 2026

Abstract

This report presents an incident-resolution assistant built during an internship at NTT DATA, designed to help engineers retrieve and act on institutional knowledge captured in past incident reports. In operational IT incident management, queries are heterogeneous: some are paraphrased descriptions of a symptom, others are exact identifiers — incident numbers, error codes, function and table names. Dense embedding retrieval handles the former well and the latter poorly, since semantically near-identical identifier strings are not separable in embedding space. The assistant addresses this with a hybrid retriever that fuses a dense sentence-embedding index with a sparse Okapi BM25 index using Reciprocal Rank Fusion (RRF), and operates under a strict privacy constraint: all inference — embeddings and generation — executes on locally-hosted models, with no external API calls.

The retriever is evaluated on a labeled benchmark of 272 queries over a corpus of 68 incident reports spanning nine technical domains. Hybrid retrieval attains $\text{Recall@5} = 1.000$ and $\text{MRR} = 0.987$, against 0.768 and 0.678 for dense-only retrieval; a per-class analysis localizes the improvement to exact-identifier queries, where dense-only Recall@5 is 0.221 against 1.000 for the hybrid. Retrieval quality is invariant to chunk size over [400,1000] characters and saturates at $k \geq 3$, a result used to bound the generation context. The retrieval component sits inside a grounded generation pipeline with intent-based abstention, adversarial-input defenses, and a post-conditioning stage that removes unattested artifacts, bounds self-reported confidence by the answer’s actual content, and flags irreversible operations the corpus does not document. A controlled comparison against a model of $2.7\times$ the parameter count found identical correctness (15/15 reports, 60/60 property checks) at 43% higher latency, supporting the practical conclusion that under a local-inference constraint, the decisive lever is context quality rather than model scale.

The system is deployed for shared, ongoing use on a data layer of independently-operated services — PostgreSQL for chat and catalog metadata, MinIO for versioned report storage, and Keycloak for OpenID Connect authentication in front of every endpoint — with the same retrieval fusion applied a second time, over the user’s own conversation history. The authentication boundary is verified by a

standing audit that probes every route in the live route table without a credential, a check reported here as targeting a defect class no unit test written against individual handlers detects. The test suite backing the project comprises 463 cases.

Contents

1	Introduction	4
2	Background	5
3	System Architecture	6
4	Platform Hardening: A Multi-Service Data Layer	8
5	Request Pipeline and Control Flow	21
6	Corpus, Representation, and Evaluation Protocol	23
7	Retrieval Method	27
8	Experimental Results	30
9	Grounded Generation	32
10	Post-Conditioning: Reconciling Answers with Evidence	34
11	Report Synthesis	36
12	Efficiency	37
13	Validation Methodology	39
14	Discussion	42

1 Introduction

1.1 Problem Setting

Incident management in large-scale IT operations produces a corpus of structured post-mortem reports: each documents a symptom, a diagnosed root cause, a sequence of resolution steps, and supporting artifacts such as queries, commands, or configuration. This corpus is institutional knowledge. Its operational value depends entirely on whether an engineer facing a new incident can retrieve the relevant prior cases quickly and reliably.

Retrieval-augmented generation [4] is the natural formulation: retrieve the most relevant prior reports, condition a language model on them, and synthesize a grounded resolution. The hard part is not generation but retrieval. Incident queries are stylistically heterogeneous, and the two dominant retrieval paradigms fail on disjoint subsets of them:

- **Dense retrieval** embeds queries and documents into a shared metric space and ranks by cosine similarity. It captures paraphrase and semantic proximity, but identifiers such as INC0012017 and INC0012018 are lexically near-identical and semantically indistinguishable; the embedding space does not separate them.
- **Sparse retrieval** (Okapi BM25 [1]) ranks by weighted term overlap. It matches identifiers and rare technical tokens exactly, but is blind to synonymy and paraphrase.

An incident assistant must serve both query classes, which motivates a hybrid retriever. The project’s guiding question was practical: how much does combining the two actually help, and on which kinds of queries does that help show up?

1.2 Deployment Constraint: Local Inference

The system is subject to a hard privacy requirement: incident reports contain sensitive operational detail and must not be transmitted to third-party inference APIs. Consequently every model — the sentence encoder and the generative model — is self-hosted, and the pipeline performs no external network calls at query time.

This constraint is not incidental; it conditions the design. It excludes frontier hosted models, imposing a lower generation-quality ceiling and a hardware-bound latency floor. It therefore shifts the engineering leverage away from model scale and toward *context quality*: what is retrieved, and how it is presented to the model. This shift is the central design principle behind the rest of the project.

1.3 Objectives and Structure

The objectives were: (i) build a retrieval pipeline that serves both semantic and lexical query classes under local-inference constraints; (ii) measure its behavior on a labeled benchmark with standard information-retrieval metrics; (iii) embed it in a generation pipeline that abstains rather than fabricates when evidence is insufficient, enforced after generation rather than by instruction alone; (iv) close the documentation loop by synthesizing new structured reports from resolved dialogues; and (v) harden the resulting system — identity, storage, and client — for use by a team rather than a single evaluator.

Section 2 reviews the retrieval concepts the design builds on. Section 3 presents the system architecture, Section 4 the multi-service data layer, authentication, and client that harden this architecture for shared use, and Section 5 the request pipeline and corpus workflow. Section 6 describes the corpus, its representation, and the evaluation protocol. Section 7 formalizes the retrieval method, including the source selection that permits abstention. Section 8 reports retrieval results. Sections 9–10 cover grounded generation and the post-conditioning that enforces its guarantees, and Section 11 report synthesis. Sections 12–13 report efficiency and the validation methodology, including the measured model-selection decision. Section 14 closes with the project’s overall conclusions.

2 Background

2.1 Retrieval-Augmented Generation

RAG [4] conditions a generative model on documents retrieved from a non-parametric memory. Formally, given a query q , a retriever selects a set $D_q \subset \mathcal{D}$ from a corpus $\mathcal{D}$, and a generator produces an answer $a \sim p(a \mid q, D_q)$. Two properties make this attractive for incident resolution: the model’s output can be traced to cited evidence, and the corpus can be updated without retraining. The critical dependency is that $p(a \mid q, D_q)$ can only be as good as D_q : retrieval failure is unrecoverable downstream, since a model conditioned on irrelevant context either hallucinates or abstains.

2.2 Sparse Retrieval: Okapi BM25

BM25 [1] scores a document d against query q as

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) (k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}, \quad (1)$$

where $f(t, d)$ is the frequency of term t in d , $|d|$ the document length, avgdl the mean document length, and $\text{IDF}(t)$ the inverse document frequency. The parameter k_1 controls term-frequency saturation and b the strength of length normalization. The inverse-document-frequency factor assigns high weight to rare terms — exactly the regime of incident identifiers, error codes, and function names.

2.3 Dense Retrieval

Dense retrieval encodes text with a transformer bi-encoder trained so that semantically related pairs are close under cosine similarity [3]. Ranking is by

$$s_{\text{dense}}(q, d) = \frac{\phi(q) \cdot \phi(d)}{\|\phi(q)\| \|\phi(d)\|}, \quad (2)$$

with ϕ the encoder. This generalizes across surface form, but its inductive bias — mapping semantically similar text to nearby vectors — is precisely what makes it unable to discriminate between distinct identifiers that are semantically equivalent (all are “an incident number”).

2.4 Rank Fusion

Combining Eq. (1) and Eq. (2) by score arithmetic is ill-posed: the two are on incomparable, unnormalized scales, and any weighting requires per-corpus calibration. Reciprocal Rank Fusion [2] avoids this by discarding magnitudes and using only rank position:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}, \quad (3)$$

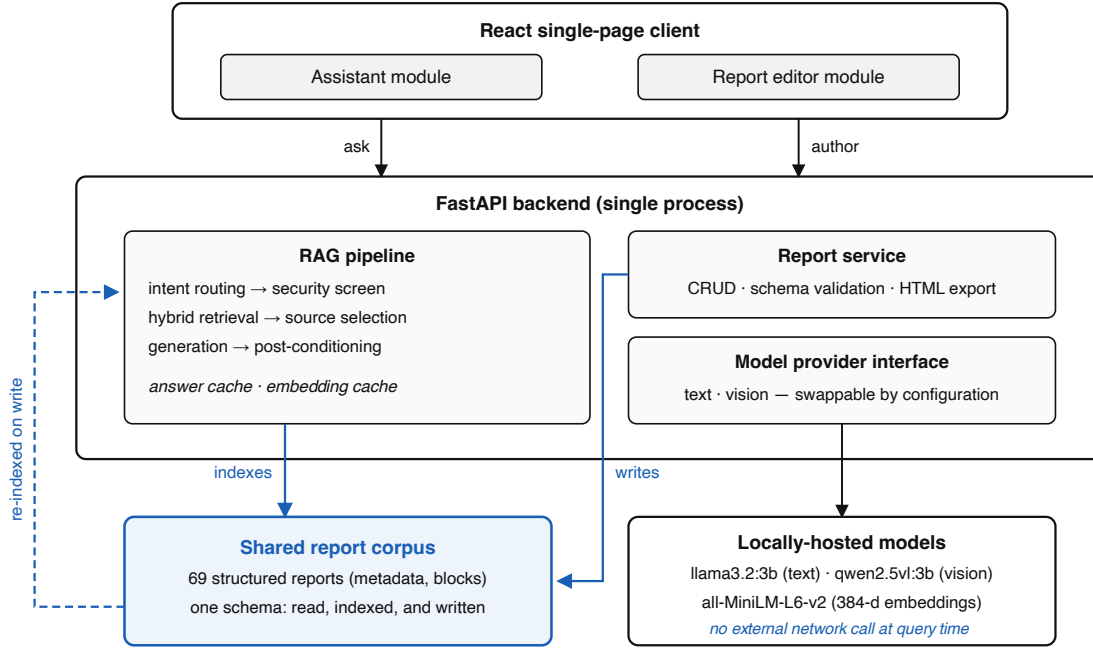
where R is the set of ranked lists and k a smoothing constant that damps the influence of top ranks. RRF is scale-free and effectively parameter-free, which is why it is adopted here.

3 System Architecture

3.1 Design Overview

The system is a single application exposing two functional modules over a shared corpus: an *assistant* that answers incident questions by retrieval-augmented generation, and a *report editor* that authors and maintains the structured reports that constitute the corpus. Both are served by one backend process and one web client.

The implementation is a modular monolith: a FastAPI backend with separated internal packages (retrieval and generation; report persistence; a shared schema and model-provider abstraction) and a React single-page client. A distributed decomposition is unwarranted — the modules share a corpus, target a single host, and have no independent scaling requirement — and would introduce network and operational overhead without benefit.



The corpus is the only coupling between the two modules — and the reason a resolved dialogue can become retrievable knowledge.

Figure 1: System architecture as first built: the web client (assistant and report-editor modules) over a FastAPI backend hosting the RAG pipeline, the report service, and the model-provider abstraction, against locally-hosted models and the shared report corpus. This is the internal component layout the retrieval and generation results of Sections 8–13 were measured against; Figure 4.1 in Section 4 shows how these same components sit behind authentication and on top of PostgreSQL and MinIO in the deployed system.

3.2 Structured Report Schema as a Shared Contract

Reports are represented by a typed schema, `Report = (metadata, blocks)`, defined once as Pydantic models and mirrored in the client’s type system. The `metadata` record carries identifying attributes (identifier, title, category, subcategory, reporter, date). The `blocks` field is a sequence over a discriminated union of content types: heading, paragraph, ordered/unordered list, code, table, image, and cross-referenced incident example.

This schema is the system’s central abstraction. It is simultaneously (i) the editor’s document model, (ii) the unit of ingestion for retrieval, and (iii) the output type of report synthesis (Section 11). A single validated contract across all three roles is what permits the assistant to both consume and produce corpus documents without translation layers.

3.3 Model Provider Abstraction

Generation is accessed through a provider interface exposing text and vision completion, with a locally-hosted implementation as the deployed backend. Decoupling the pipeline from a concrete model permits the generative model to be selected by configuration, and isolates the retrieval and orchestration logic from model-specific concerns. The deployed configuration uses a 3B-parameter instruction-tuned model for text and a 3B vision-language model for screenshot analysis; sentence embeddings use `all-MiniLM-L6-v2`, a 384-dimensional encoder.

3.4 Deployment

The application is containerized with a multi-stage build in which the client is compiled and served as static assets by the backend process, yielding a single-process deployment. Persistent state comprises the report corpus, the embedding-model cache, and dialogue history; Section 4 describes the multi-service data layer that holds it and the authentication in front of it.

The container runs the application under an unprivileged account rather than as `root`. Because named volumes are created root-owned at mount time — which masks ownership assigned during the image build — an entry point assumes ownership of the mounted paths and then replaces itself with the application process under the unprivileged identity. Replacing rather than forking matters: the application becomes process 1 and therefore receives termination signals directly, so shutdown is orderly (measured at one second, exit status zero) instead of terminating by timeout, which would abandon in-flight response streams and risk a torn database write. A health probe reports readiness distinctly from liveness, allowing an orchestrator to restart a process that is running but unable to serve. Memory is bounded so that a fault in this container cannot exhaust the host it shares with the inference server.

4 Platform Hardening: A Multi-Service Data Layer

Sections 8–13 evaluate the retrieval and generation stack described above. This section documents the platform the stack runs on: a data layer built from established, independently-operated services, authentication in front of every endpoint, retrieval extended to the dialogue history itself, and a client built to the standard expected of an internal tool. A system intended for shared, ongoing use requires a verified identity on every request and a database that tolerates concurrent writers — properties distinct from the retrieval quality that Sections 8–13 measure.

The retrieval and generation stack of Sections 3–10 operates unchanged on this platform: what this section describes is everything the stack sits on and everything a user touches before reaching it.

4.1 Requirements and Scope

Three properties are required of a system serving more than one user. *Identity* must be verified rather than asserted, so that access can be granted at different levels to different people and every action is attributable. *Persistence* must tolerate concurrent writers, retain version history on a report edited after its incident closes, and support search across accumulated conversations. The *interface* must organize a growing conversation list, persist user preferences, and present the report-authoring module and the assistant — which share a corpus — as one system.

The scope of the platform work follows from these three requirements: identity and access control (Section 4.2); durable, queryable storage for chat history and report artifacts (Section 4.4); the attachment and upload surface (Section 4.3); retrieval extended from the report corpus to the user’s own conversation history (Section 4.5); and a client rework covering navigation, theming, and the report-authoring surface (Section 4.6). Each of the four is deployed as a separate, independently-operated container, chosen deliberately over

folding the same capability into the existing FastAPI process — the justification for that choice is given alongside each service below.

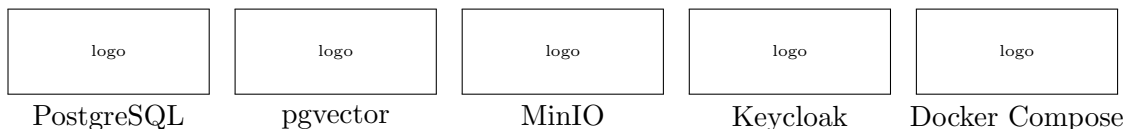


Figure 2: The hardened architecture. The client authenticates against Keycloak before any request reaches the backend; the backend verifies the resulting token on every route. The RAG pipeline, report service, and model provider are otherwise the same components described in Section 3, now reading and writing through PostgreSQL and MinIO instead of a single SQLite file and a bind-mounted directory.

4.2 Identity and Access Control

Authentication is delegated to Keycloak, an OpenID Connect identity provider, rather than implemented in the application. The application never observes a password: the client performs an Authorization Code flow with PKCE (Proof Key for Code Exchange) directly against Keycloak, and the backend’s only responsibility is verifying the resulting signed token against Keycloak’s published key set on every request. Three realm roles are defined — **admin**, **analyst**, **viewer** — mapped to write access, read-and-write access to the assistant and report editor, and read-only access respectively; role assignment is a Keycloak administrative action, not an application code path.

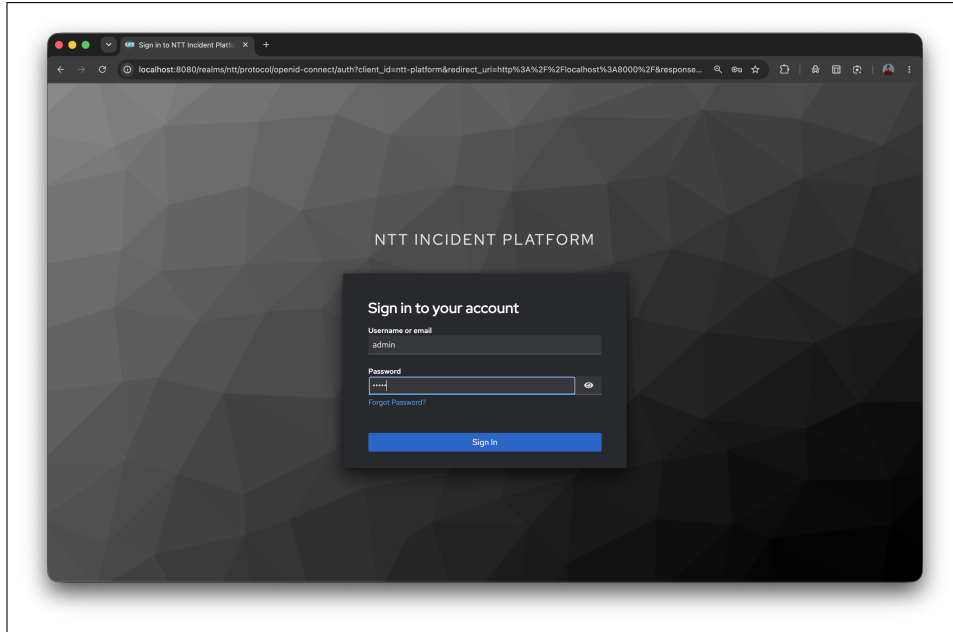


Figure 3: The Keycloak-hosted sign-in screen, reached from the application’s own login page. The application redirects here and never sees the submitted credentials; only a signed token is returned.

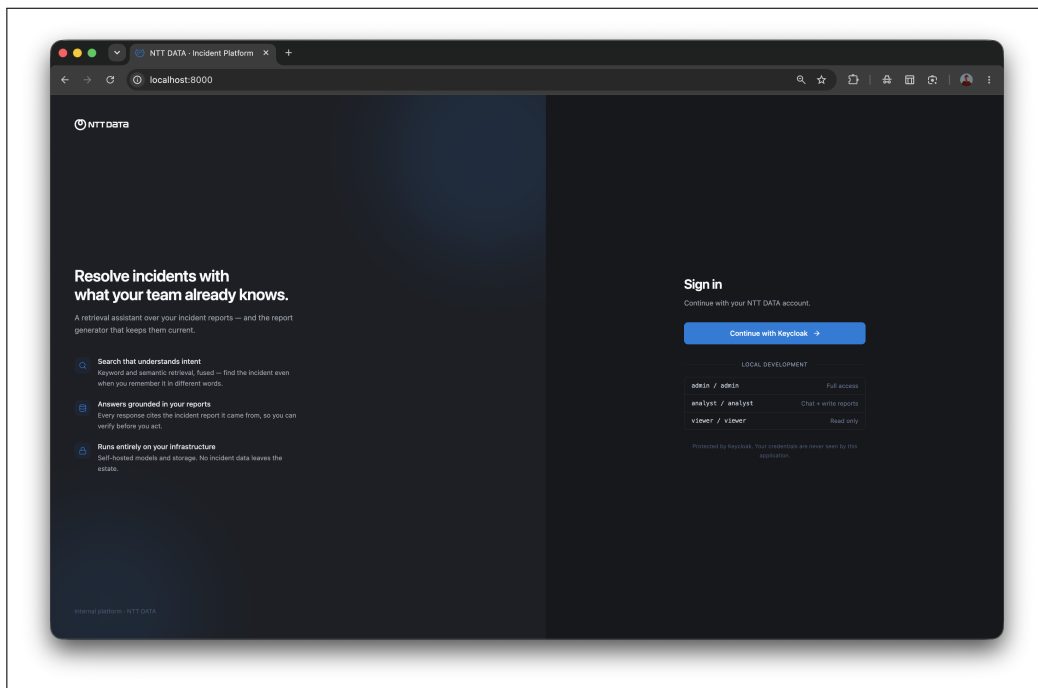


Figure 4: The application’s sign-in page. A single action starts the OIDC flow; local development accounts are listed for convenience since this deployment is not internet-facing.

Coverage of the token check is verified by an automated audit rather than by inspection: every route in the application’s route table is probed without a token, and any response other than an explicit rejection fails the suite. The audit runs against the live route table, so an endpoint added later is covered without the test being updated.

This check is load-bearing — it is the kind of defect a route-by-route code review does not surface, since each handler can be written correctly against an assumption about where in the pipeline authentication happens, and only an adversarial probe tests that assumption. The report-deletion endpoint is the clearest case: it carries exactly the destructive-operation risk that Section 10 guards against in generated *content*, and it must be guarded equally as an *API*.

4.3 Attachment Handling and the Upload Surface

Accepting a file from a user is conventionally one of the more dangerous capabilities a web application can expose: the archetypal risks are an uploaded executable or archive that is later run or expanded, a path-traversing filename that escapes its intended directory, and a document whose declared type differs from its true contents. The system’s exposure to these is bounded structurally rather than by inspection of what is uploaded, and the argument is worth stating explicitly because it rests on what the application does *not* do.

There is no general file-upload endpoint. The application defines no multipart route and never receives a file as a file; the two surfaces that accept binary data both accept it as base64 text inside a JSON request body, and each is constrained separately.

Screenshot attachments to the assistant are the larger of the two. Attached image data is bounded on arrival and passed to the vision model as an opaque string. It is never decoded, never parsed by an image library, never given a filename, and never written to disk; no code path exists by which the bytes could be executed, expanded, or placed in the filesystem. An archive or an executable submitted through this field is therefore not a file that the system mishandles — it is a string that fails to describe an incident, and the model’s inability to read it is the whole of its effect. The client restricts selection to image types, but that restriction is a usability affordance rather than a control: it is enforced in the browser and cannot be relied upon, which is why the property that matters is the absence of a decoding path on the server rather than the presence of a filter on the client.

Profile images are the second surface, and are treated more strictly because their disposition differs: an avatar is stored and then rendered back into the page for every viewer of the application. That makes an unvalidated value a stored cross-site-scripting vector rather than merely bad input — a data URI declaring a document type instead of an image would execute in the browser of anyone who loaded the page. Submissions are therefore accepted only against an explicit allowlist of image types, with the payload required to decode as valid base64 and bounded in size, and anything else is rejected at the API boundary.

Report filenames, which reach the system through save and delete operations rather than through an upload, are guarded independently: every path-taking operation rejects directory separators and parent-directory references before a filename is used, and the object-storage layer applies the same rule a second time to storage keys. The guard is applied at the service boundary rather than per handler, for the reason given in Section 4.2 — a control placed on individual routes protects only the routes that remember it.

Two limitations bound this argument honestly. The application enforces no global request-body size limit, deferring that to the reverse proxy in front of it, which is conventional but means the protection is a deployment responsibility rather than an application-level guarantee. And the size ceilings on attached images and stored avatars limit resource consumption without constituting content inspection: nothing verifies that a well-formed

image is benign, only that it is bounded and, where it will be rendered, that it is genuinely an image.

4.4 Data Layer: PostgreSQL and MinIO

Two service boundaries replace the single SQLite file and the bind-mounted report directory.

Chat and catalog metadata move to PostgreSQL. The migration is schema-driven and versioned (Alembic), and the store’s public interface is preserved exactly across the change, so the request pipeline of Section 5 required no modification — only its persistence dependency changed. Bringing dialogue history under a relational schema, rather than treating it purely as an opaque blob, is also the precondition for Section 4.5: a query language over the conversation store is what makes it searchable at all.

Report artifacts move to MinIO, an S3-compatible object store, addressed by content-derived keys with the Postgres report-catalog table holding metadata and pointing at the corresponding object. Versioning is enabled on the bucket, so an edit to an authored report retains the prior version rather than discarding it — relevant because a report is commonly revised after its incident closes. The corpus (68 reports at evaluation time, Table 1) is held in this store, each object verified present and byte-identical on entry.

Two properties of this storage layer are worth stating, both established by verifying moved data against its source rather than by inspecting the code that moves it. Object keys accommodate the identifiers the corpus actually contains: eighty of its ninety-three report filenames carry spaces and punctuation, so a key derivation assuming alphanumeric identifiers is insufficient, and the derivation is validated by diffing every object across the full corpus rather than a sample. The uniqueness constraint is keyed on the storage object rather than on the incident identifier, because the corpus contains reports filed against an identifier already in use, and keying on the identifier silently discards the second.

4.5 Retrieval Over Conversation History

The retrieval method of Section 7 operates over the report corpus. The same fusion — dense embedding similarity combined with sparse lexical matching by Reciprocal Rank Fusion (Eq. 3) — is applied a second time, over a different corpus: the user’s own prior conversations, stored as described in Section 4.4. This is implemented as the identical fusion function applied to a second pair of rankers rather than a parallel reimplementa- tion, for the same reason the underlying method was chosen for report retrieval in the first place — a query for a past conversation is subject to the same semantic/lexical split as a query against the report corpus: a user recalling a conversation by its gist needs the dense ranker, and a user recalling a specific term or identifier needs the sparse one.

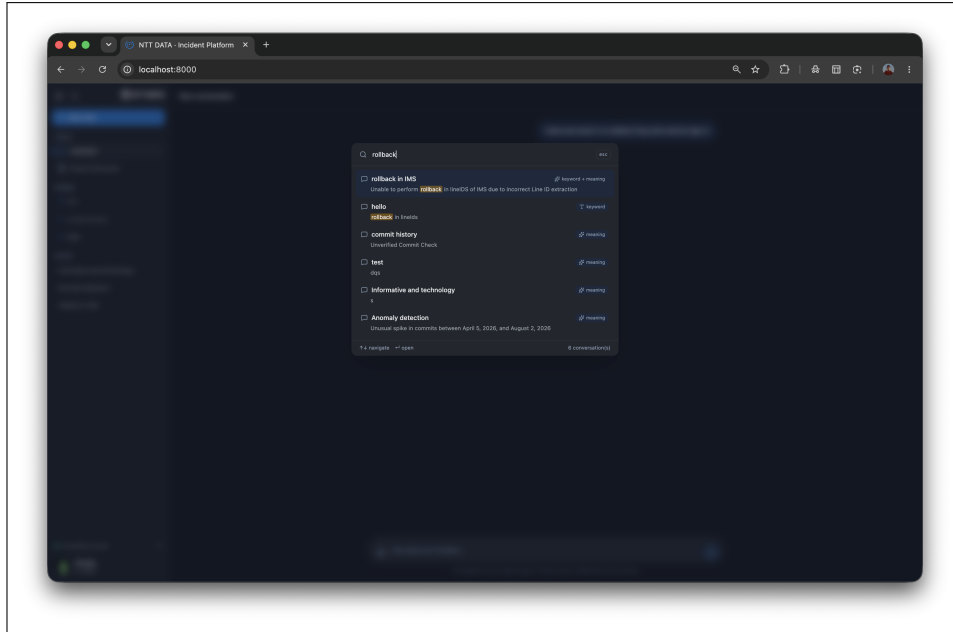


Figure 5: Conversation search. Each result is labeled by which ranker matched it (*keyword*, *meaning*, or *both*); the top result here matched by keyword and by meaning simultaneously.

Two further capabilities follow from the same relational chat store: conversations may be renamed and pinned, and an administrative view aggregates the answer-quality feedback (thumbs and free-text corrections) already collected by the mechanism of Section 9.4, which existed as a data-collection path with no corresponding interface until this phase.

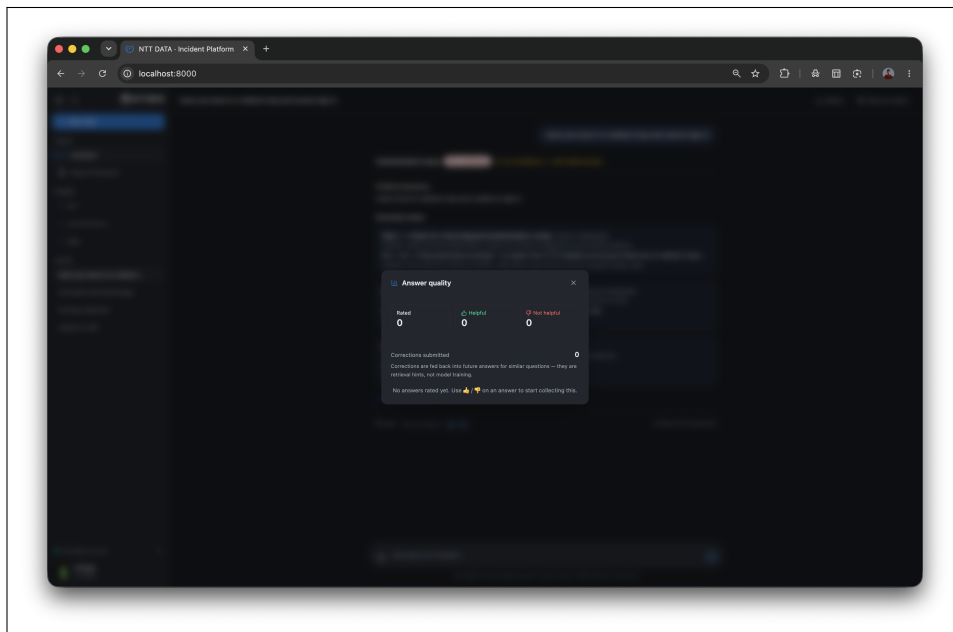


Figure 6: Aggregated answer-quality metrics, visible to administrators. The underlying thumbs and correction data were already collected (Section 9.4); this view is the first consumer of it.

4.6 Client Rework

The client was restructured around the navigation model the identity and search work now supports: a persistent sidebar listing pinned and recent conversations, a tools switcher between the assistant and the report editor, and a settings surface exposing three color themes (a light theme and two dark variants distinguished by contrast level) plus per-user composing preferences, all persisted per account rather than per browser.

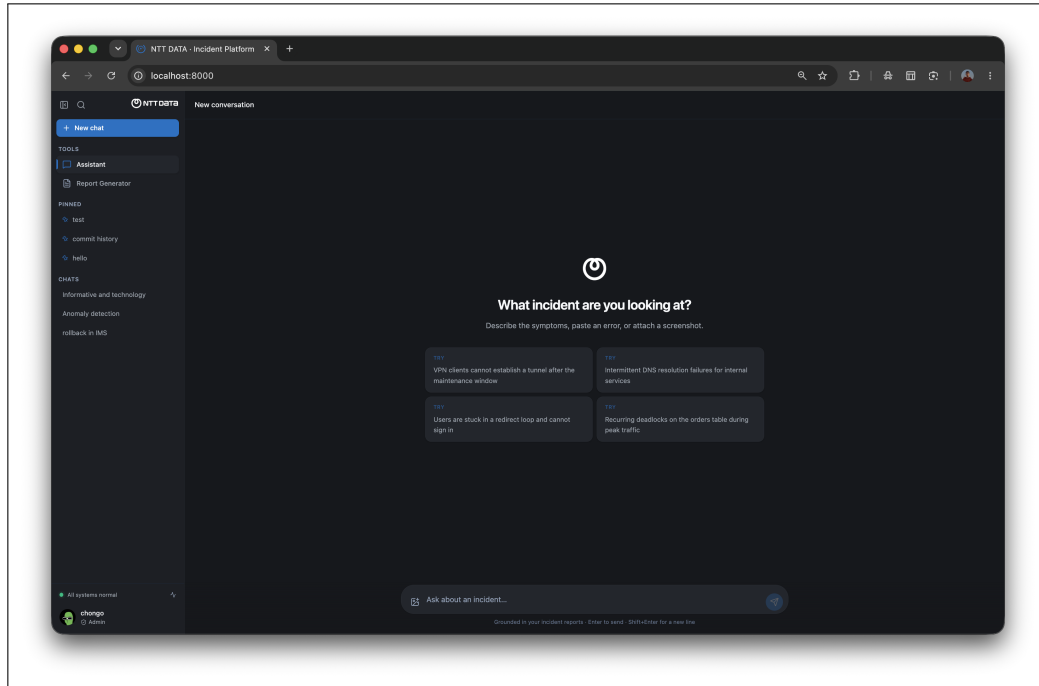


Figure 7: The assistant, signed in, showing the pinned/recent conversation sidebar, the tool switcher, and a suggested-query empty state.

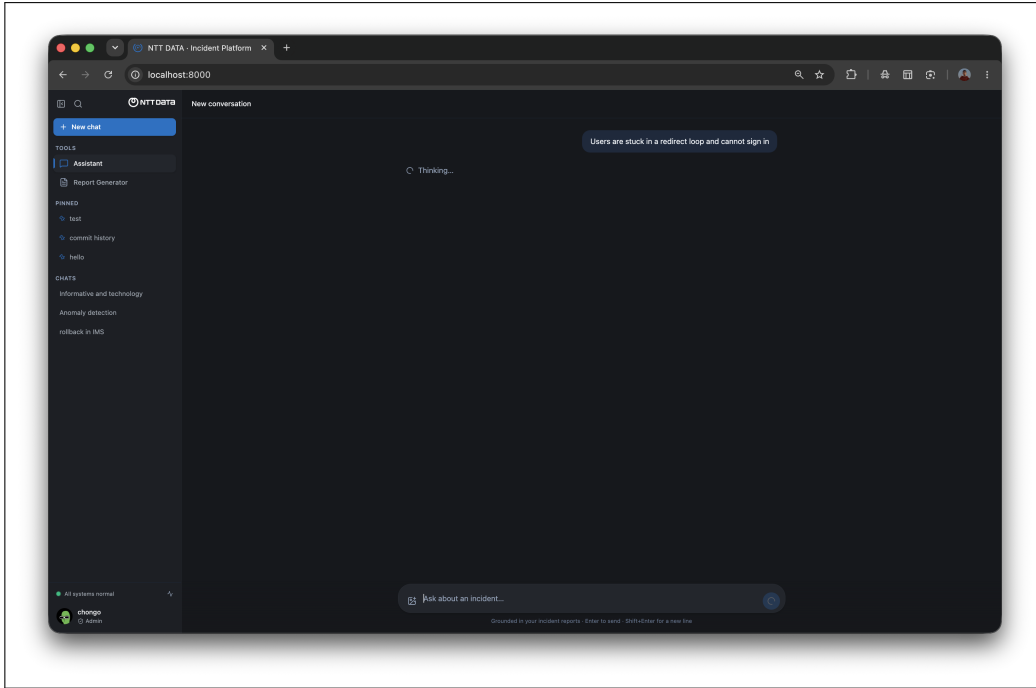


Figure 8: The interface while generation (stage 7 of the pipeline in Section 5) is in flight, distinguishing an answer being composed from one already returned rather than presenting a blank wait.

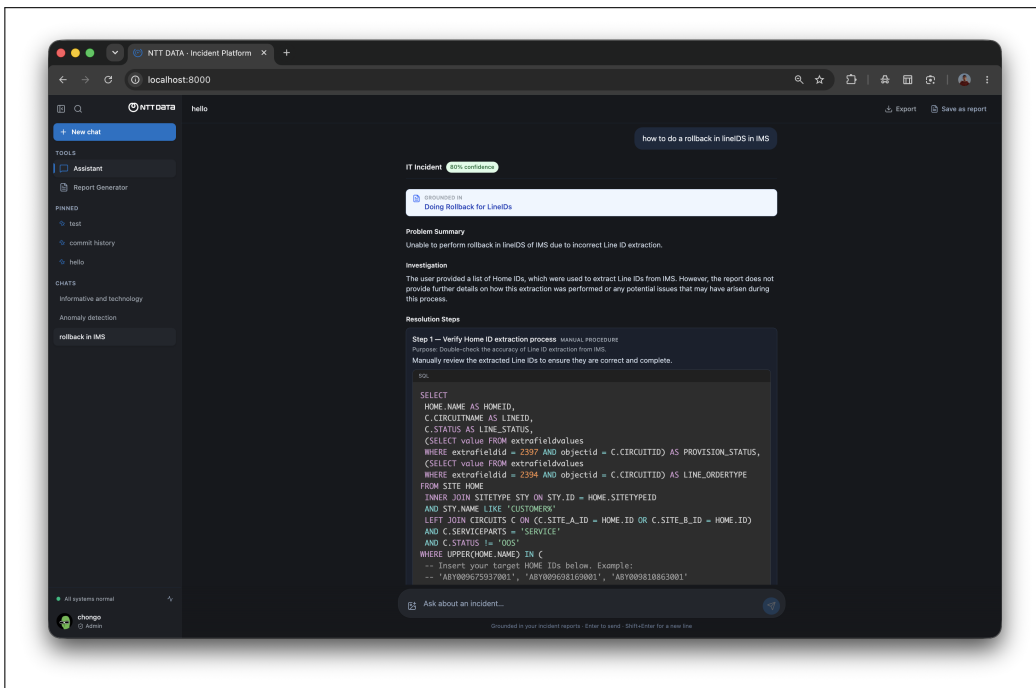


Figure 9: The completed, grounded answer: the cited source report, a confidence badge, and an executable SQL artifact drawn from that report — the same pipeline output described in Section 9, now in its production interface.

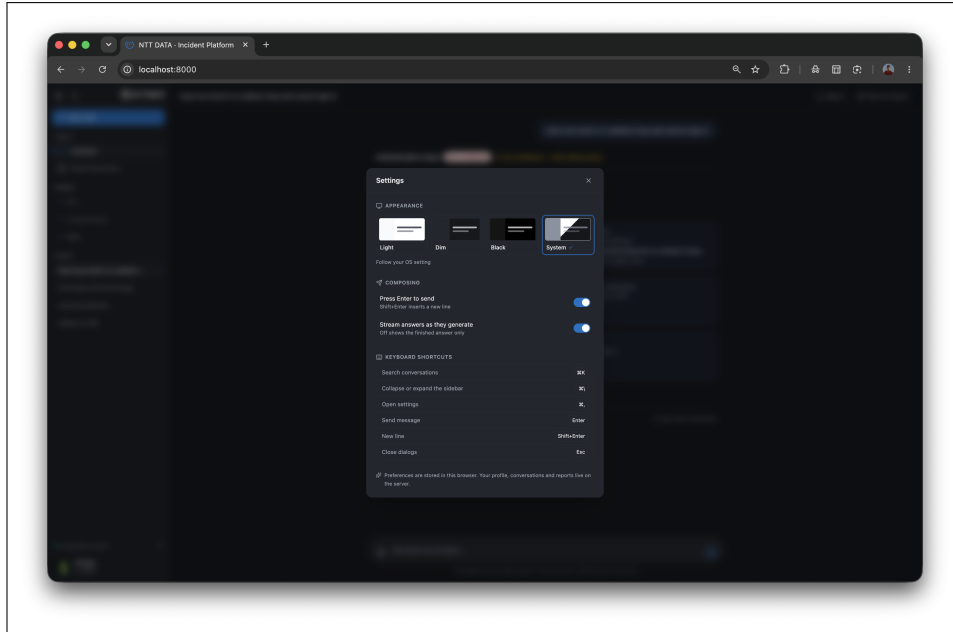


Figure 10: Theme and composing preferences, persisted server-side per account.

4.7 Query Formulation Aids

Two additions address a property of the pipeline established in Section 9: answer quality is bounded by question specificity, because retrieval operates on the query text and abstention is triggered by inputs carrying no diagnosable content. Both therefore act on the question rather than on the answer.

The first is a set of composer commands. Typing `/` opens a filtered list of templates — root-cause analysis, resolution procedure, the exact query used for a fix, validation of a proposed remediation, and retrieval of comparable incidents — and selecting one expands it into the composer with the substantive segment preselected, so that typing replaces it directly. The templates are deliberately incomplete: each expands to a question requiring the user to supply the incident, since a command such as *root cause* carries no meaning until the symptom is named. This is client-side text expansion with no server component and no additional prompt path; it steers the user toward the query shape the existing pipeline handles best rather than introducing a second answer mode.

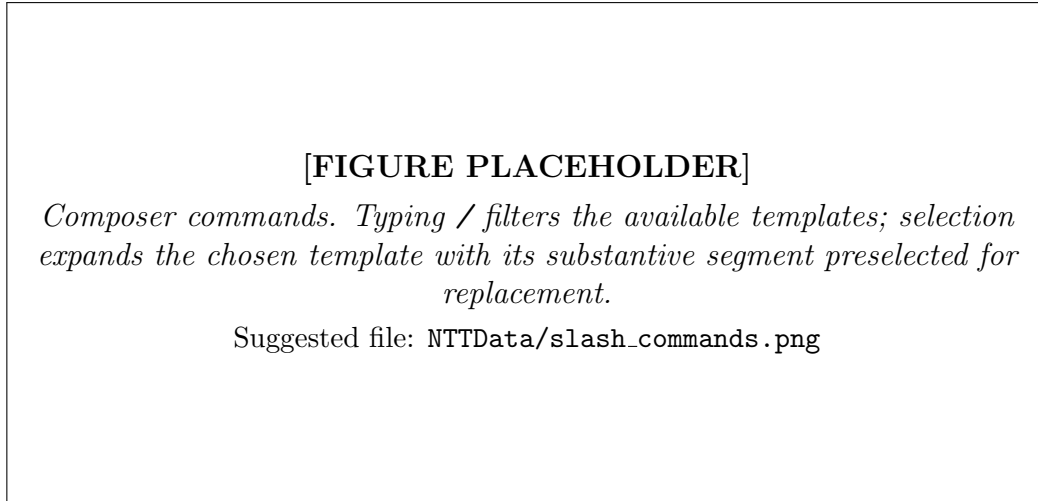


Figure 11: Composer commands. Typing / filters the available templates; selection expands the chosen template with its substantive segment preselected for replacement.

The second is follow-up suggestion. On request, the system proposes three short questions continuing from a returned answer, generated by a separate and much smaller prompt conditioned on a compressed form of that answer. Two design constraints govern it. It is invoked on demand rather than automatically, so the latency of the primary path (Section 12) is unchanged for users who do not use it; and it is implemented as an isolated call with its own cache rather than as an extension of the resolution prompt, so the post-conditioning guarantees of Section 10 are neither modified nor re-derived. Malformed model output degrades to no suggestions rather than to an error.

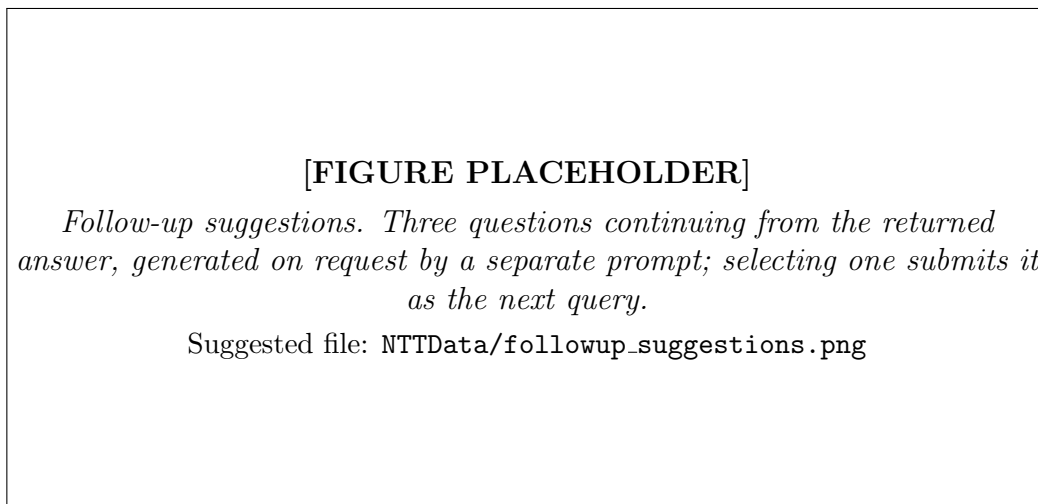


Figure 12: Follow-up suggestions. Three questions continuing from the returned answer, generated on request by a separate prompt; selecting one submits it as the next query.

4.8 Viewport Adaptation

The client adapts its layout to the viewport. At desktop widths a fixed-width sidebar sits beside the content area, with a collapse control that reduces it to an icon rail. Below a tablet breakpoint that arrangement does not hold — a fixed sidebar of usable width consumes the majority of a handset screen — so the sidebar becomes an overlay panel,

dismissed by selection or by tapping outside it, and the content area occupies the full viewport. Header controls reduce to their icons at narrow widths. The layout is verified by measurement at four viewport widths rather than by inspection, confirming at each both the content width available to the application and the absence of horizontal overflow.

The report editor shares the assistant’s visual treatment, as the two modules share the corpus described in Section 6 and read as one system. Report listing, search and filtering, and the block-based content editor all sit on the shared design system. This is a presentation concern only: the **Report** = (**metadata**, **blocks**) schema of Section 3.2 and the synthesis mechanism of Section 11 are independent of it, and write through the same interface regardless.

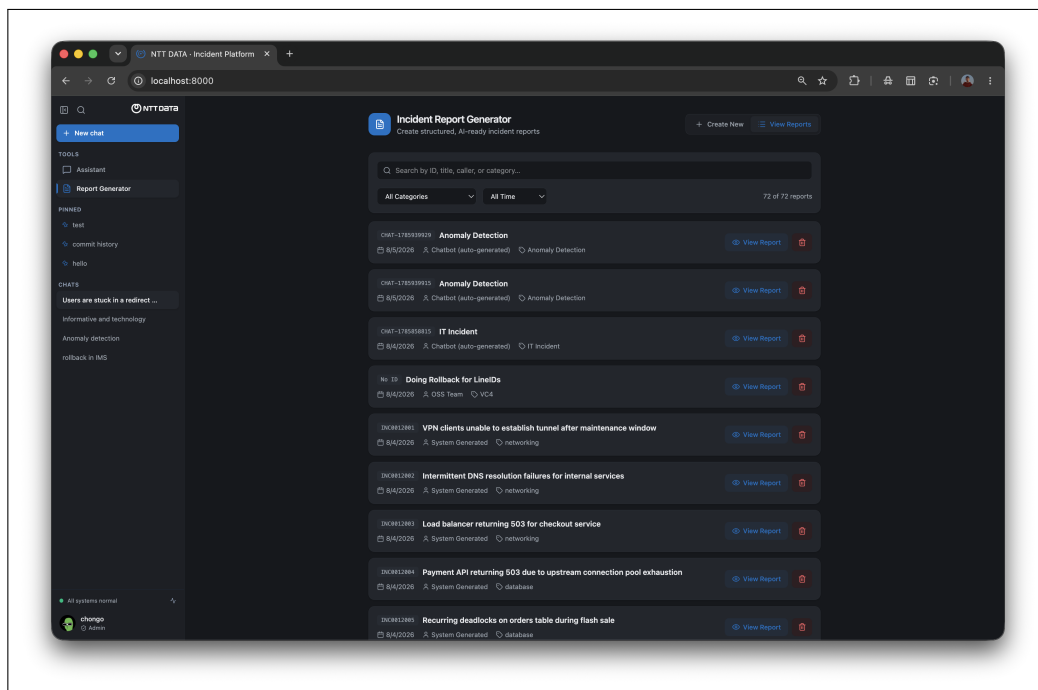


Figure 13: The report editor’s list view: search, category and date filters, and per-report actions, now on the same design system as the assistant.

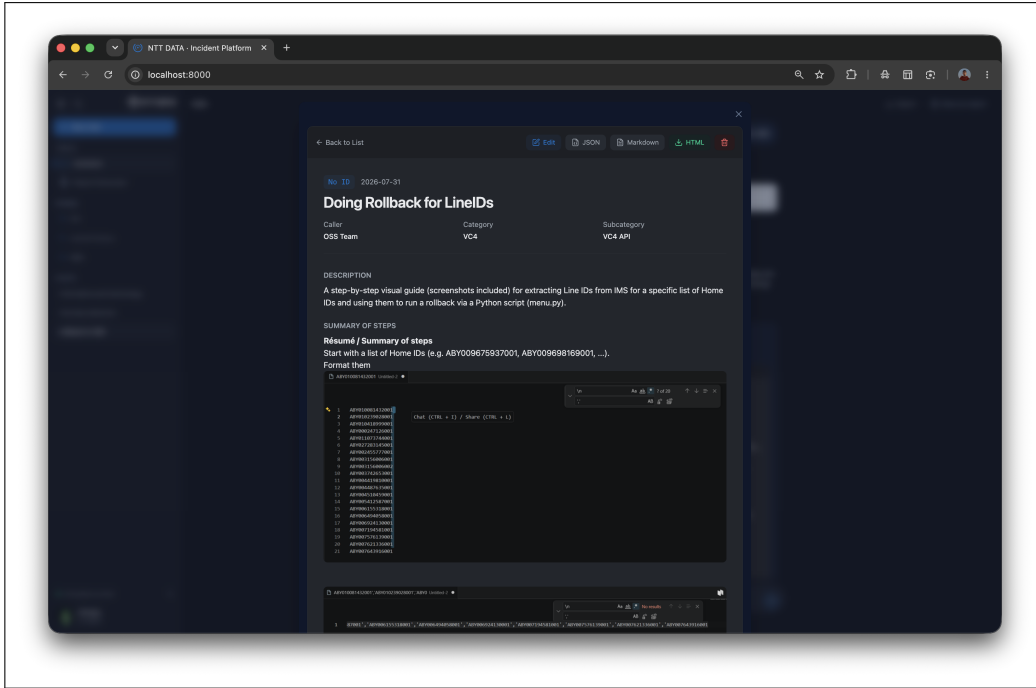


Figure 14: A synthesized report, opened in the viewer, with its export actions (JSON, Markdown, HTML) and edit entry point.

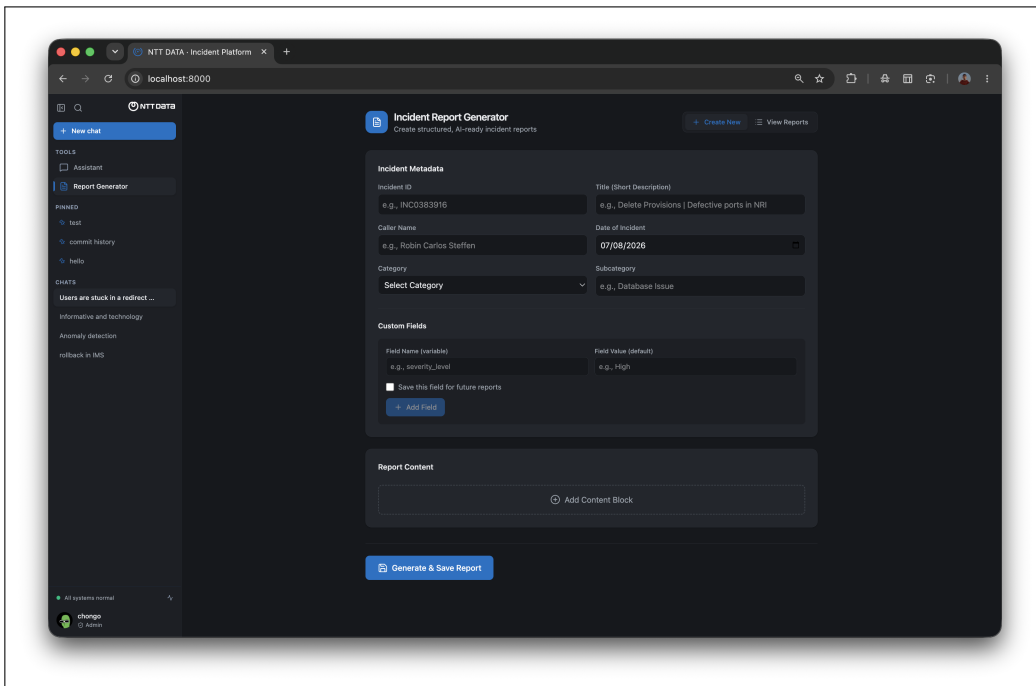


Figure 15: Report authoring: the metadata form of Section 3.2 and, below it, the empty block-content editor for a new report.

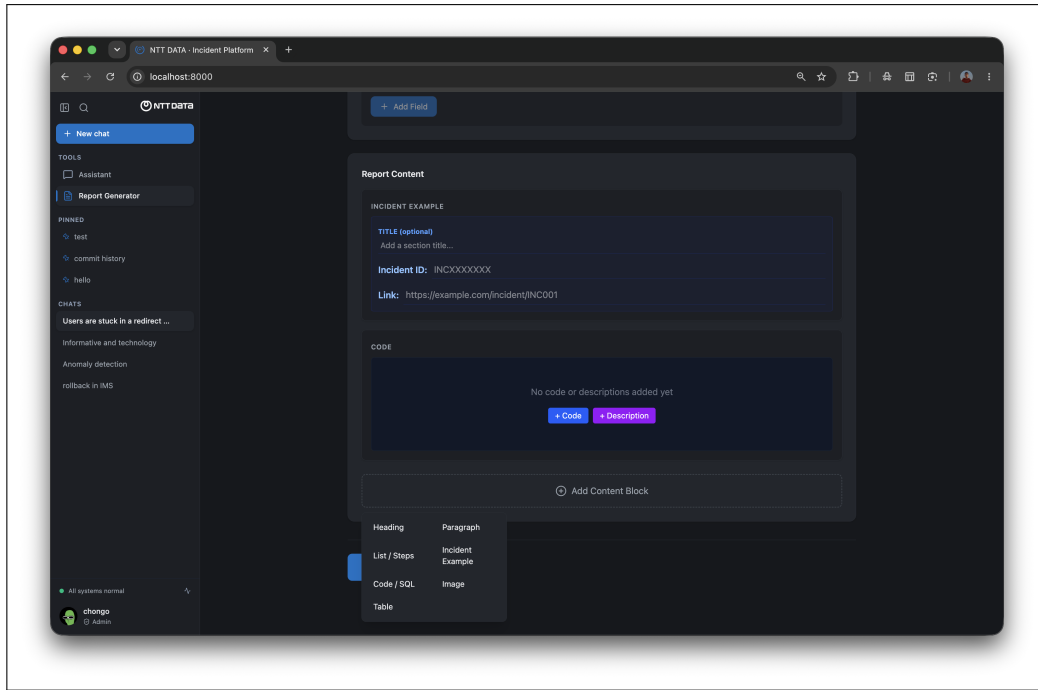


Figure 16: The block-type picker, mid-authoring: each of the seven block types of Section 3.2 — heading, paragraph, list, incident example, code, image, table — is directly selectable, and existing custom fields and a code block are already attached to this report.

Two further points close the loop between identity and the rest of the client. Per-account display name and avatar are editable independently of the identity fields Keycloak owns (username, email, role), which are read-only in this view precisely because they are not this application's data to write.

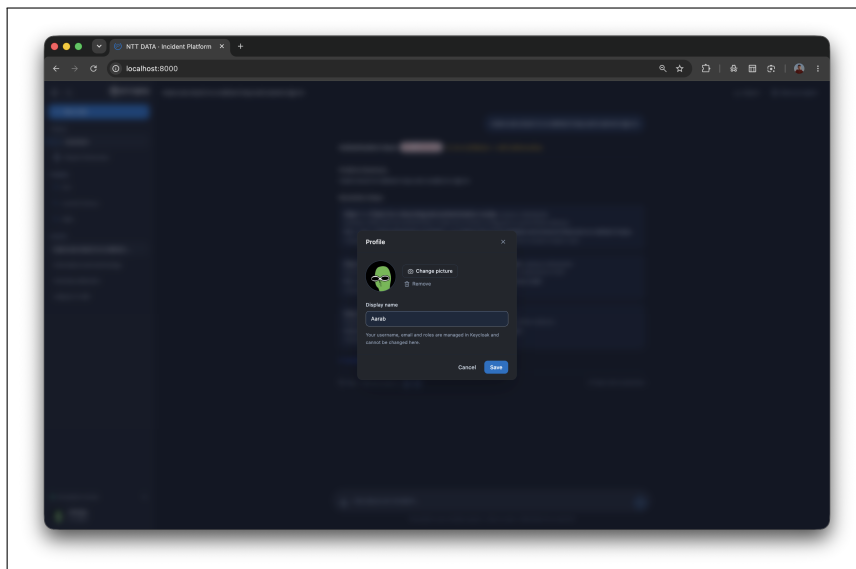


Figure 17: Per-account profile: display name and avatar are stored by the application; username, email and role are Keycloak-owned and shown read-only.

4.9 Verification

The component-test discipline of Section 13 extends to the platform layer: every migration script asserts against the true source of record before that source is retired, the authentication audit of Section 4.2 is a standing test rather than a one-time check, and the full suite of 463 cases is required to pass before any container image is rebuilt. Coverage spans the data migration, the authentication boundary, and the conversation-search ranking in addition to the retrieval and generation behavior of Section 13.

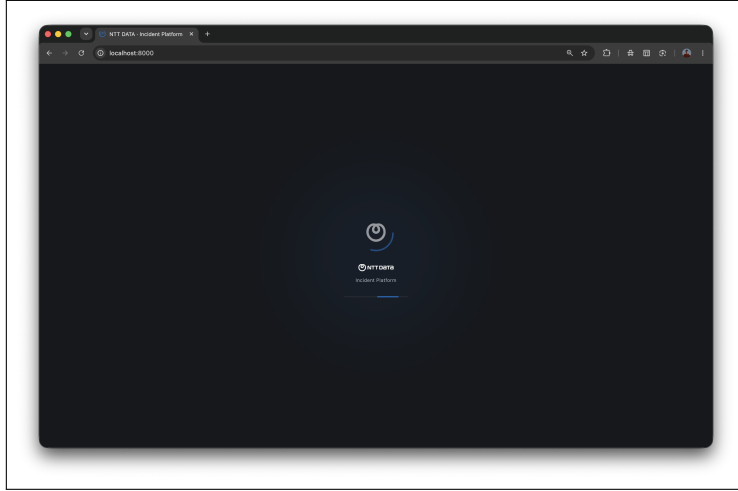


Figure 18: The application’s loading screen, shown while the session and its permissions are resolved against Keycloak before the interface renders.

5 Request Pipeline and Control Flow

The architecture of Section 3 describes what the components are. This section describes how a request moves through them, because the ordering of stages — not merely their presence — is what produces the behavior evaluated in Sections 8 and 10.

5.1 Staged Execution with Early Termination

A request is processed as an ordered sequence of stages, each of which may terminate the request and return. The design principle is that a stage which can rule out an answer should run before any stage that could fabricate one. Cheap, decisive checks therefore precede expensive, permissive ones.

1. **Intent routing.** The turn is classified as conversational, out-of-scope, under-specified, or diagnosable. Only the last proceeds; the others are answered directly, without retrieval or generation. This is the structural guarantee of Section 9: fabrication on non-diagnosable input is impossible because no generator is invoked.
2. **Adversarial screening.** Text is scanned for instruction-override and extraction patterns and enclosed in an untrusted-data delimiter (Section 9).
3. **Query construction.** The query is combined with prior dialogue turns and, where an image is supplied, with a vision-model description of it, forming the retrieval query.

4. **Hybrid retrieval.** Dense and sparse indices are queried and fused by Reciprocal Rank Fusion (Section 7).
5. **Source selection.** Candidates are de-duplicated by source document, re-scored by entity overlap with the query, and filtered by an absolute and a relative threshold (Section 7.3). An empty result here is a valid outcome and suppresses citation entirely.
6. **Conditioning.** Selected documents are supplied *in full* rather than as the matched fragment, together with any applicable stored corrections.
7. **Generation.** The model emits a structured object under constrained JSON decoding.
8. **Post-conditioning.** The parsed answer is reconciled against the evidence: unsupported citations and artifacts are removed, confidence is bounded by what the answer actually contains, and irreversible operations are annotated (Section 10).

Stages 1 and 5 are the two points at which the system declines to answer, and both precede generation. Stage 8 is the only stage that can weaken an answer after the model has produced it, and it never strengthens one — a deliberate asymmetry, since every check it applies is a check against over-claiming.

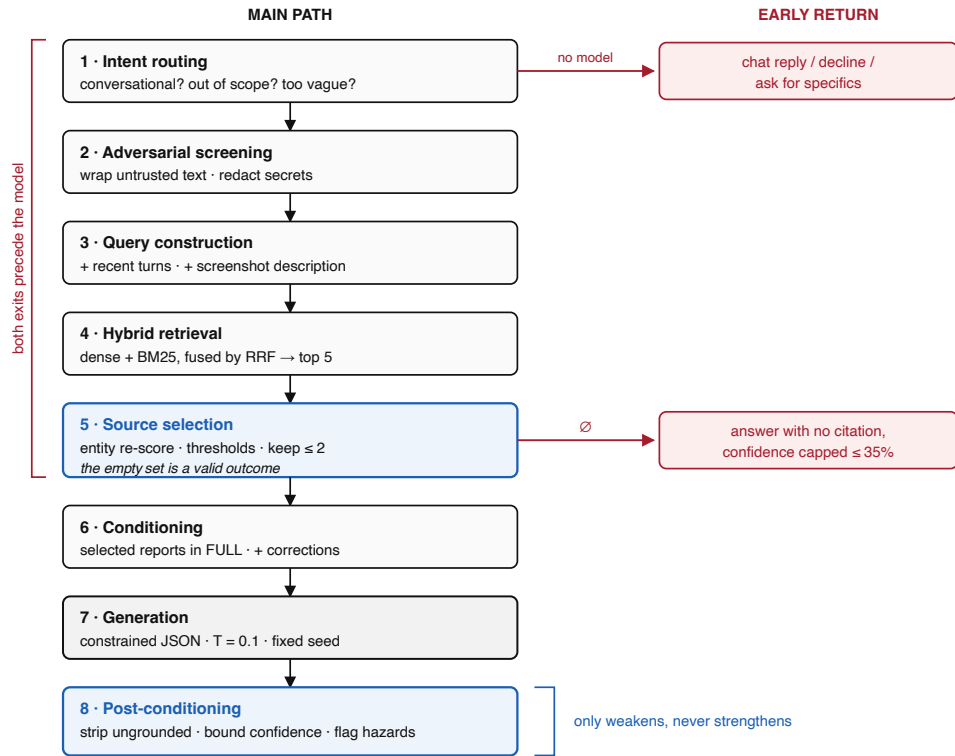
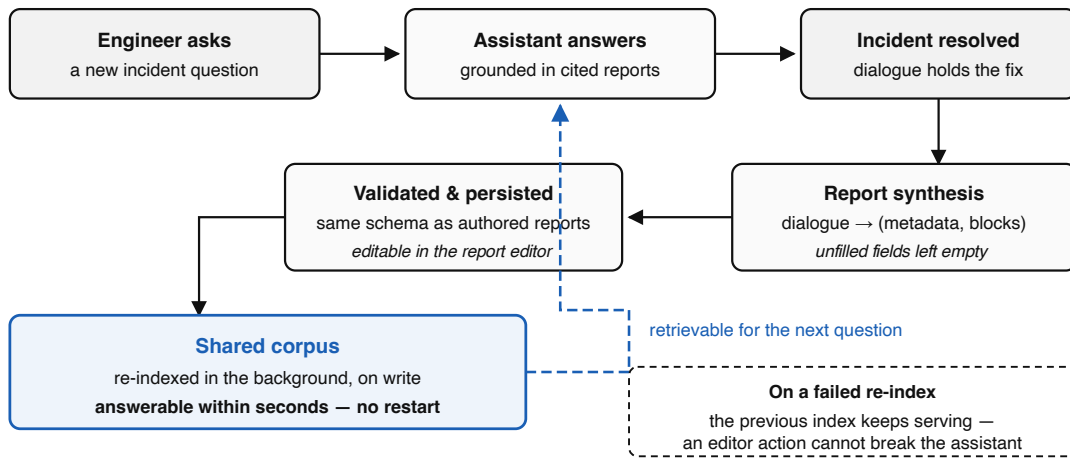


Figure 19: Request pipeline: intent routing and adversarial screening precede retrieval; hybrid retrieval feeds source selection, which may return the empty set; selected documents condition generation; post-conditioning reconciles the answer against the evidence before it reaches the client.

5.2 Corpus Maintenance Workflow

The two modules share the corpus, so a write by the editor must become visible to the assistant. Writing a report or deleting one triggers re-indexing as a background task: the response returns immediately and the index is rebuilt without the author waiting for embedding. The rebuilt index replaces the live one only on success, so a failed re-index leaves the previous working index serving requests rather than degrading the assistant as a side effect of an editor action. Cached answers are discarded at the same moment, since they were conditioned on evidence that has changed.

This closes the operational loop implied by Section 11: a resolution synthesized from a dialogue is answerable within seconds of being written, with no restart, so the corpus grows during an incident rather than after it.



The corpus grows during an incident rather than after it: each resolution becomes evidence for the next.

Figure 20: Corpus maintenance: a report written or deleted through the editor triggers background re-indexing and cache invalidation, making it retrievable by the assistant without a restart.

6 Corpus, Representation, and Evaluation Protocol

6.1 Corpus

The corpus comprises 68 incident reports spanning nine technical domains: networking, databases, authentication, infrastructure, frontend, messaging and queueing, continuous integration and delivery, monitoring, and security. Each report carries domain-characteristic failure signatures — HTTP status codes, message-broker error codes, container exit codes, TLS handshake alerts, DNS resolution failures.

The corpus is constructed to be discriminative rather than merely representative. It contains two families of near-miss pairs designed to defeat shallow matching:

- **Shared symptom, distinct root cause:** e.g. two incidents presenting as HTTP 503 from connection-pool exhaustion, one caused by a connection leak in an unrelated workload, the other by an undersized pool at peak concurrency.

- **Shared error code, distinct context:** e.g. container exit code 137 arising from node-level memory-pressure eviction versus a batch process exceeding its own memory limit.

A retriever that keyword-matches on 503 or 137 cannot separate these; correct ranking requires discriminating on context.

Table 1: Corpus statistics (source: evaluation notebook, cell 3).

Property	Value
Reports	68
Distinct categories	30
Median tokens per report	147.5
Token range per report	[103, 175]
Block-type frequency	
Paragraph	148
Heading	89
Code	72
List	68
Table	20
Incident example	12
Description box	8
Image	7

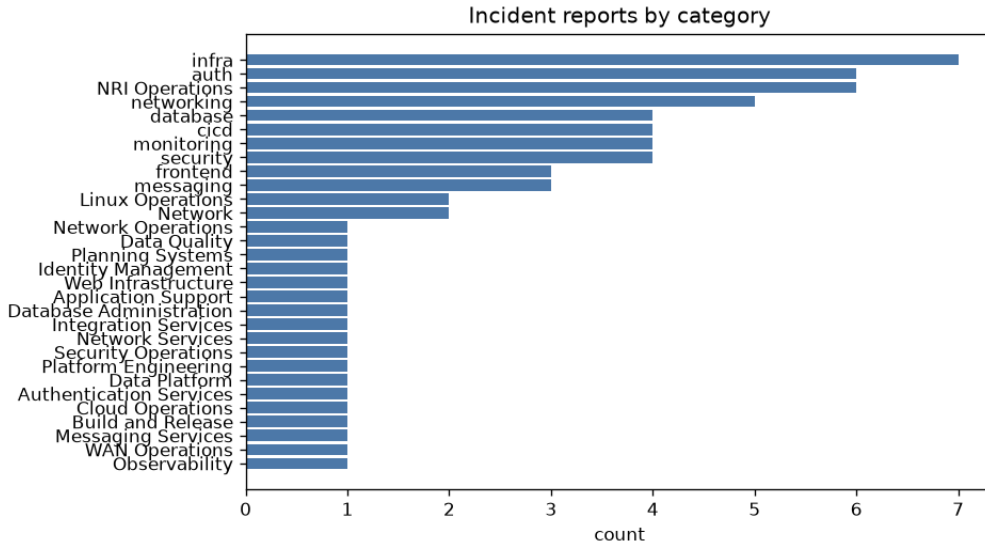


Figure 21: Category distribution over the corpus (source: notebook cell 3).

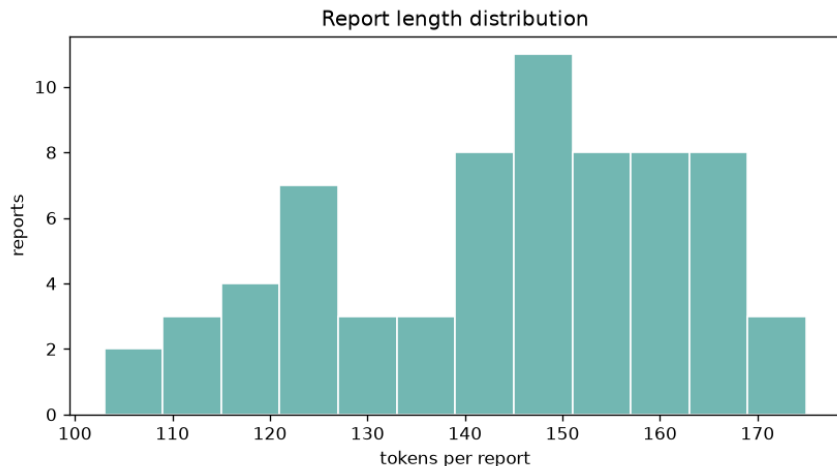


Figure 22: Report length distribution in tokens (source: notebook cell 3).

6.2 Document Representation

Reports are stored as structured documents, not free text, so the indexed representation must be derived by a projection $\pi : \text{Report} \rightarrow \Sigma^*$. The choice of π is consequential and easy to get wrong.

A naive projection serializes the document tree verbatim, emitting structural tokens — block identifiers, type discriminators, nesting keys, encoded image payloads — interleaved with content. The resulting string is dominated by tokens carrying no incident semantics. Both stages degrade: the encoder ϕ maps such strings into a region governed by shared structural vocabulary rather than incident content, collapsing inter-document distances; and BM25’s IDF weighting is diluted by high-frequency structural terms. Downstream, the generator is conditioned on text in which the actual symptom and resolution are a minority of tokens.

The system therefore defines π *schema-aware*: it dispatches on block type and emits only the semantic payload of each — heading text, symptom and root-cause prose with markup stripped, resolution list items, code content, table cells — while discarding identifiers, type tags, and binary data. Metadata is emitted as a leading header so that identifiers and category remain lexically searchable. This projection is a precondition for the retrieval results of Section 8: no amount of ranking refinement compensates for an uninformative representation.

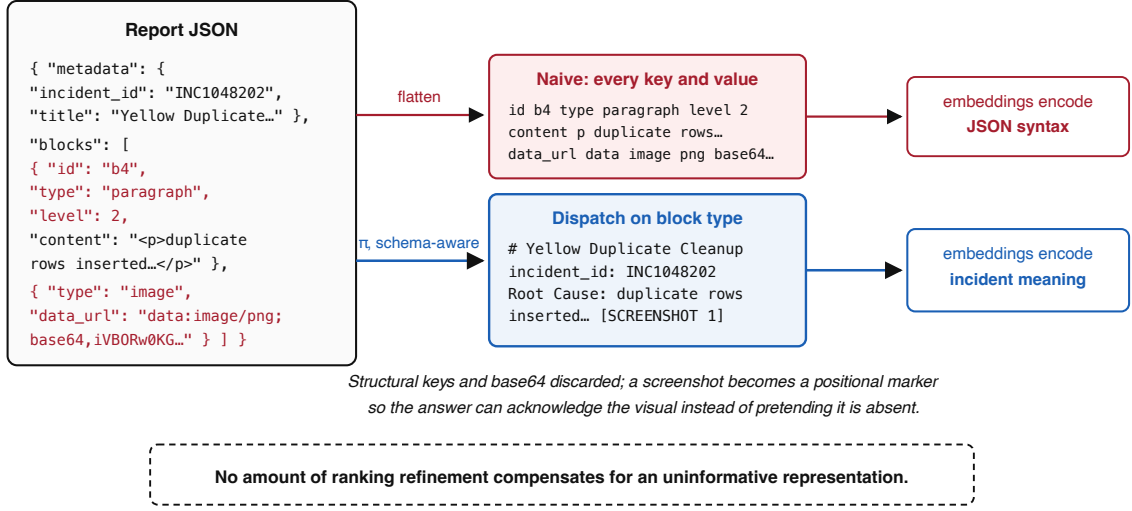


Figure 23: Document representation. The naive projection emits structural keys and base64 payloads, so the encoder embeds JSON syntax; the schema-aware projection π dispatches on block type and emits only incident content, replacing an inline screenshot with a positional marker.

6.3 Chunking and Indexing

Projected documents are segmented into overlapping character windows (700 characters, 120-character overlap) to bound the encoder’s input and to localize matching. Each chunk is encoded by ϕ into $\mathbb{R}^{384}$ and simultaneously tokenized into the BM25 index. Retrieval operates over chunks; results are aggregated to the report level by taking each source document’s maximum-scoring chunk, so a report is ranked by its single best-matching passage.

6.4 Evaluation Protocol

Evaluation requires query–document relevance labels. Absent production query logs, labels are constructed by inversion: for each report d , queries are synthesized that a user seeking d plausibly issues, with d as the gold document. Four stylistic classes are generated, chosen to span the semantic–lexical axis that motivates the hybrid:

- **title** — an interrogative form of the report title (semantic, high signal);
- **symptom** — the root-cause sentence (semantic, paraphrastic);
- **id** — the bare incident identifier (purely lexical);
- **keyword** — salient technical tokens such as function and table names (lexical, moderate).

This yields 272 labeled queries over the 68 reports. Stratifying by class is essential: an aggregate score would mask the fact that the two retrieval paradigms fail on *disjoint* query classes, which is the phenomenon under study.

Validity. The queries are synthetic, derived from the documents they retrieve, so the protocol measures recovery of a document from a paraphrase of its own content; real queries are noisier and may under-specify, so the reported figures should be read as an upper bound on field performance. The protocol is still informative for its purpose: it is a *comparative* instrument, and because dense, sparse, and hybrid are measured under identical conditions, the relative ordering and the per-class localization of error hold even where the absolute values are optimistic.

Metrics are Recall@ k (whether the gold report appears in the top k), Mean Reciprocal Rank, and nDCG@5, computed with a single relevant document per query.

7 Retrieval Method

7.1 Components

Three configurations are compared under identical chunking and identical corpus:

1. **Dense:** cosine ranking by Eq. (2) using `all-MiniLM-L6-v2`.
2. **Sparse:** BM25 by Eq. (1) with standard parameters $k_1 = 1.5$, $b = 0.75$. The index is implemented directly rather than by dependency, which keeps the retrieval stack self-contained and its behavior fully specified — relevant under a constraint that forbids external services. Tokenization preserves underscores, so composite identifiers such as `support_remove_opv_duplicate` remain single high-IDF terms rather than fragmenting into common subwords.
3. **Hybrid:** RRF over the dense and sparse rankings by Eq. (3), with $k = 60$.

7.2 Fusion

Each component produces a ranking over chunks. Documents scoring zero under BM25 are excluded from its list rather than appended arbitrarily, so that non-matching documents contribute no reciprocal-rank mass. The fused score of Eq. (3) is then aggregated per source report as described above, and the top K reports are returned.

`TOP_K = 5` reports are retrieved. These are candidates, not citations: what reaches the generator and the user is determined by the selection stage below, which Section 8.3 shows can be restricted well below K without recall loss.

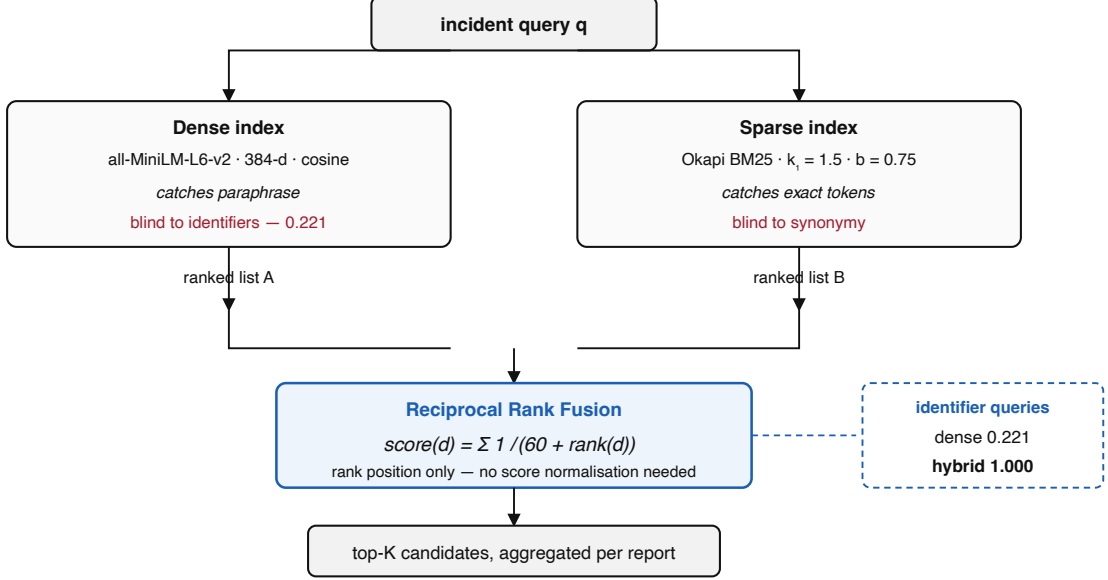


Figure 24: Retrieval architecture: a query is encoded and simultaneously matched against the dense index and the BM25 index; the two rankings are fused by RRF and aggregated per source report.

7.3 Source Selection

Ranking is not the same as relevance. RRF always returns its top- K candidates, ordered, whether or not any of them answers the query — the fusion score is a rank statistic, and a rank exists even when nothing matches. A pipeline that cites the top of that list unconditionally will attribute an answer to whichever document happened to rank first, which is worse than returning nothing: it lends a real incident’s authority to a guess.

A selection stage therefore intervenes between fusion and generation. Let H be the fused candidates and $E(q)$ the set of content terms in query q after stop-word removal. For a candidate h with retrieval score $s(h)$, define

$$\sigma(h) = s(h) + \lambda \cdot \frac{|E(q) \cap E(h)|}{|E(q)|}, \quad (4)$$

where $E(h)$ draws on the candidate’s title and body, with title matches weighted double, and λ scales the lexical contribution. The weighting reflects that RRF scores occupy a narrow range — they are reciprocals of small integers — so entity overlap, not fusion score, is what separates a document that *mentions* a term from one that *is about* it.

Candidates are de-duplicated by source report, ranked by σ , and admitted subject to two thresholds:

$$\sigma(h) \geq \tau_{\text{abs}} \quad \text{and} \quad \sigma(h) \geq \rho \cdot \max_{h' \in H} \sigma(h'), \quad (5)$$

with at most M reports retained. The relative threshold $\rho = 0.85$ prevents a single strong match from being padded with materially weaker ones. The absolute threshold $\tau_{\text{abs}} = 0.50$ is the one that permits the empty set, and it is what makes “nothing in the corpus documents this” an expressible outcome rather than an unreachable branch.

Its value is calibrated from the separation between question classes on this corpus, which is wide and unambiguous. Every incident question drawn from the corpus scores $\sigma \approx 1.04$; questions on subjects the corpus does not cover peak at 0.29 (Table 2). A threshold placed between them rejects the latter with margin on both sides while leaving every genuine match untouched — a property asserted directly by the test suite of Section 13, so that tightening τ_{abs} later cannot silently begin refusing questions the corpus does answer.

Table 2: Selection scores by question class. The threshold $\tau_{\text{abs}} = 0.50$ lies in the gap.

Class	Example	σ
Documented incident	“how to rollback lineIds”	1.042
Documented incident	“kafka consumer group lag growing”	1.042
Documented incident	“public API certificate expired”	1.042
Out of corpus	“configure the mainframe COBOL scheduler”	0.282
Out of corpus	“how do I tune a guitar”	0.288
Out of corpus	“how do I file my income taxes”	0.160
Out of corpus	“best recipe for sourdough bread”	0.029

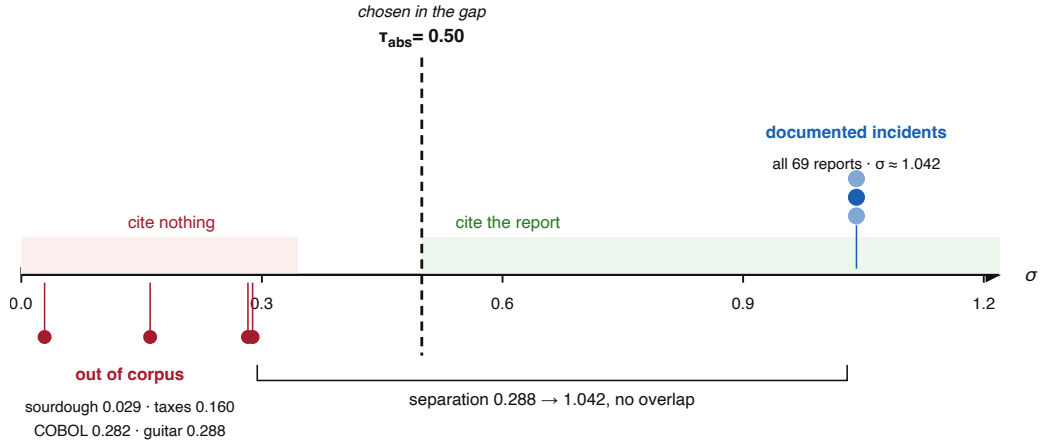


Figure 25: Source selection. Every documented incident question scores $\sigma \approx 1.042$; questions outside the corpus peak at 0.288. The absolute threshold is placed in the resulting gap, so a question the corpus does not answer yields no citation at all.

Selection caps the admitted set at $M = 2$ reports, and each is supplied to the generator *in full* rather than as the matched chunk. This inverts the usual context-budget trade-off: rather than many fragments from many documents, the generator receives the complete text of the few documents that genuinely match. A documented procedure is frequently a multi-step sequence — extract with a query, run a script, verify the result — and a chunk boundary that severs it yields an answer that is correct about the first step and silent about the rest.

8 Experimental Results

All quantitative results are produced by the evaluation notebook (`eval/evaluation.ipynb`); each table cites its source cell.

8.1 Aggregate Retrieval Quality

Table 3 reports aggregate performance over the 272-query benchmark. The hybrid dominates both components on every metric, attaining $\text{Recall@5} = 1.000$ and $\text{MRR} = 0.987$ against 0.768 and 0.678 for dense retrieval.

Table 3: Aggregate retrieval performance, 272 labeled queries (source: notebook cell 6).

Configuration	R@1	R@3	R@5	MRR	nDCG@5
Dense	0.629	0.713	0.768	0.678	0.700
Sparse (BM25)	0.779	0.982	0.989	0.872	0.902
Hybrid (RRF)	0.974	1.000	1.000	0.987	0.991

Two observations merit attention. First, sparse retrieval is the stronger single component on this corpus ($\text{Recall@5} = 0.989$ versus 0.768), which is consistent with a domain whose documents are dense in rare, discriminative technical tokens; this is a corpus property, not a general claim about BM25. Second, and more informative, the hybrid’s advantage over BM25 is concentrated at the top rank: Recall@1 improves from 0.779 to 0.974 while Recall@5 improves only from 0.989 to 1.000. The dense signal is therefore acting chiefly as a *re-ranking* prior — BM25 places the gold document in the candidate set, and the semantic signal promotes it to rank one. This is the mechanism by which the fusion earns its gain on this corpus.

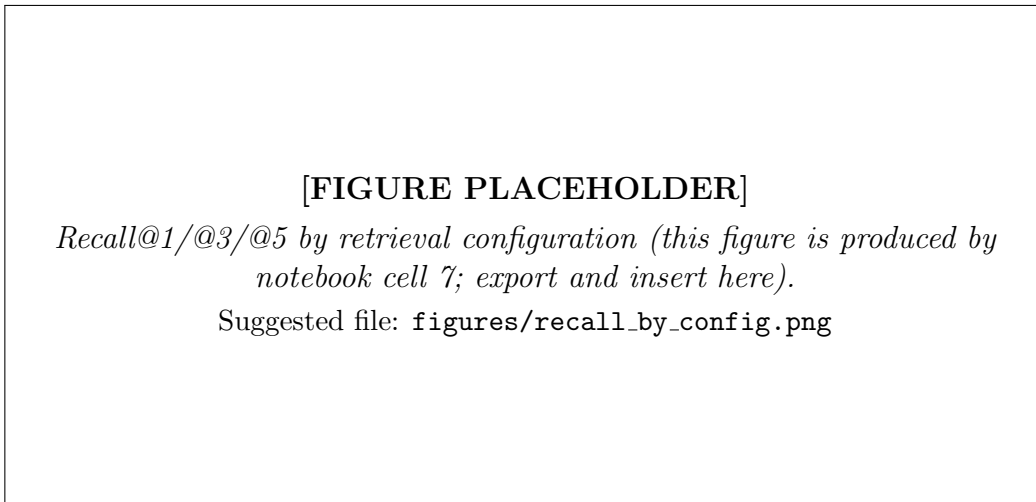


Figure 26: Recall@1/@3/@5 by retrieval configuration (this figure is produced by notebook cell 7; export and insert here).

8.2 Per-Class Analysis

The aggregate figures conceal the phenomenon of interest. Table 4 stratifies Recall@5 by query class and localizes the entire dense/sparse asymmetry.

Table 4: Recall@5 stratified by query class (source: notebook cell 6).

Query class	Dense	Sparse	Hybrid
id (identifier)	0.221	1.000	1.000
keyword	0.853	0.956	1.000
symptom	1.000	1.000	1.000
title	1.000	1.000	1.000

On **symptom** and **title** queries — paraphrastic and semantically rich — dense retrieval is already saturated at 1.000; the sparse component contributes nothing there, and critically, costs nothing either. On **identifier** queries the dense retriever collapses to 0.221. This is not a tuning deficiency but a consequence of the encoder’s inductive bias: all incident identifiers occupy a narrow region of the embedding space because they are semantically the same object, so cosine similarity cannot rank the correct one above its near-neighbors. BM25’s IDF term, which assigns near-maximal weight to a token appearing in exactly one document, resolves these exactly (1.000).

The hybrid attains 1.000 on all four classes. It does not merely average the components’ performance — it attains the upper envelope, because RRF requires only that *one* constituent rank the gold document highly, and the two components’ failure modes do not overlap. Their complementarity is what makes the fusion effective; had both failed on the same queries, no rank fusion could recover them.

8.3 Ablations

Two parameters are ablated on the hybrid configuration (Table 5).

Table 5: Hyperparameter ablations, hybrid configuration (source: notebook cell 10).

(a) Chunk size		
Size (chars)	Chunks	Recall@5
400	334	1.000
700	180	1.000
1000	146	1.000
(b) Retrieved k		
k	Recall@5	MRR
1	0.974	0.974
3	1.000	0.987
5	1.000	0.987
10	1.000	0.987

Chunk size. Recall@5 is invariant at 1.000 across [400, 1000] characters, though the index size varies by a factor of 2.3 (334 to 146 chunks). The invariance follows from the corpus geometry: at a median of 147.5 tokens (Table 1), most reports are shorter than even the smallest window, so segmentation rarely splits a report’s discriminative content across chunks. Chunking would become a live hyperparameter on a corpus of

longer documents; on this one it is not, and reporting it as insensitive is more useful than reporting a tuned value that does not generalize.

Retrieved k . Recall@5 saturates at $k = 3$ (0.974 at $k = 1$, 1.000 for $k \geq 3$). This licenses a design decision with direct cost consequences: the generation context is bounded to the top three reports rather than five. Since the marginal reports contribute no additional recall, the truncation is lossless with respect to retrieval while reducing prompt length — which, under local inference, is the dominant controllable term in generation cost. The ablation converts an arbitrary constant into a justified one.

8.4 Error Analysis

The hybrid configuration records **0 failures over 272 queries** (0.0%), with empty failure-by-class and failure-by-cause distributions (notebook cell 13). No query fails to place its gold report in the top five.

A zero error rate should be read against the protocol’s stated bias (Section 6): with queries synthesized from the documents they retrieve, and a corpus of 68 documents rich in discriminative tokens, the task is easier than field retrieval. What the result does establish is that the retriever is not the limiting component under these conditions, and that the near-miss pairs deliberately embedded in the corpus are correctly separated.

9 Grounded Generation

9.1 Conditioning and Output Structure

Retrieved reports are formatted into a prompt with their similarity scores and identifiers, and the model is constrained to emit a structured JSON object: an incident classification, a confidence score, an ordered sequence of resolution steps each with a purpose, action, validation criterion, and evidence references, and typed supporting artifacts. Constrained JSON decoding removes conversational preamble and makes parsing deterministic. Generation length is capped, bounding worst-case latency.

Artifacts are typed by language rather than assumed. The correct artifact for an incident is domain-dependent — a shell command for a missing file, a manifest fragment for a container misconfiguration, a query for a data defect — and the model selects the language accordingly. Fixing the artifact type a priori (for instance, always emitting SQL) produces confidently-formatted but semantically irrelevant output on incidents outside that domain.

9.2 Abstention Under Insufficient Evidence

A generator instructed to always produce a resolution will produce one regardless of evidence, which converts absent information into fabricated content. The pipeline therefore admits abstention at two points.

First, before retrieval, an intent classifier routes each turn. Inputs that are conversational, or that are incident-flavored but carry no diagnosable content (no error code, identifier, file path, service name, or quoted message), are answered with a request for specifics. Because this precedes both retrieval and generation, fabrication on under-specified input is structurally impossible rather than discouraged. Follow-up turns are exempt, as prior dialogue supplies the missing context.

Second, the generation prompt permits the model to declare insufficient evidence and lower its confidence rather than emit a fixed number of steps, and forbids citing identifiers absent from the retrieved set. The system also distinguishes a model backend failure from a low-confidence answer: an unreachable model raises an explicit error rather than returning a well-formed but empty result, which would otherwise be indistinguishable from a genuine negative diagnosis.

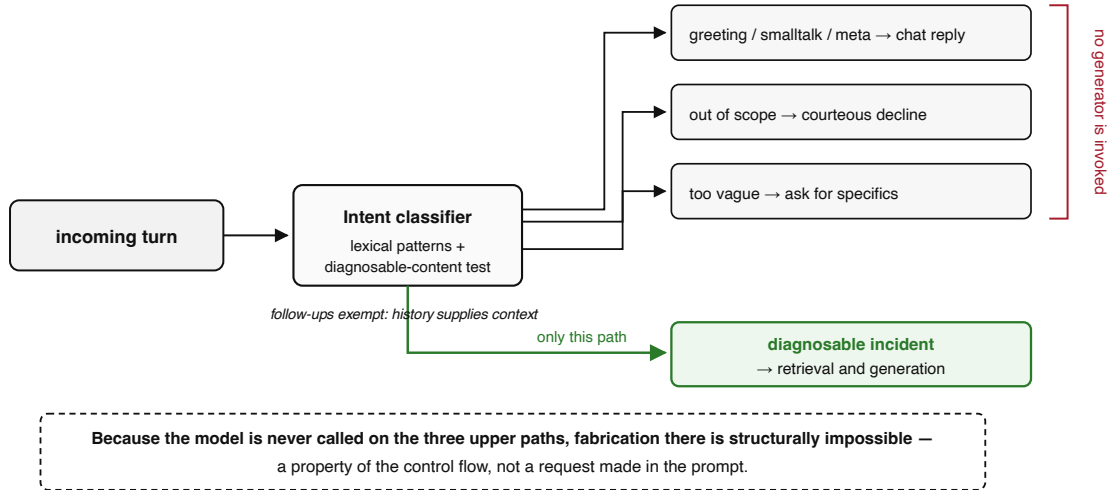


Figure 27: Intent routing and abstention. Three of the four outcomes are answered without invoking a generator at all, so fabrication on those paths is a property of the control flow rather than an instruction in the prompt.

9.3 Adversarial Input

User text is screened for instruction-override and prompt-extraction patterns; detected inputs are enclosed in an explicit untrusted-data delimiter within the prompt, so the model treats them as content to analyze rather than instructions to follow, and the system prompt is never emitted. A legitimate incident containing such phrasing is still processed as an incident, with the detection surfaced to the user, rather than refused. Secrets and personal data (credentials, bearer tokens, connection strings, private keys, email addresses) are redacted from dialogue text before persistence.

9.4 Feedback

Answers may be rated, and a negative rating admits a free-text correction. Corrections are stored and, on subsequent queries with lexical overlap to the corrected question, injected into the prompt as high-priority guidance. The mechanism is explicitly a retrieval-time intervention over an inspectable store, not parameter adaptation: no model weights are modified, and the system makes no claim to the contrary.

Ratings also condition source selection. Each stored answer records the reports it cited, so the net rating accruing to a report is derivable from data the system retains anyway, without additional schema. During selection (Section 7.3), a report’s combined score receives a bounded adjustment proportional to its net rating. The bound is deliberate and small: feedback orders reports that already match, and does not determine

whether they match. Ratings are aggregated for the administrative view of Section 4.5 in addition to this use.

The ordering of the two operations carries the safety property. The adjustment is applied *after* the absolute relevance floor, never before it. A report that fails the floor is discarded on topical grounds and is not reconsidered, so no accumulation of positive ratings can promote an unrelated report into the cited set — the property holds by construction rather than by choice of constant, and is asserted directly in the test suite at ratings several orders of magnitude beyond any realistic value.

The interface states the effect of each control, on the principle that a feedback affordance implying an effect it does not have is worse than none. A positive rating reports that the cited source will be preferred for similar questions. A negative rating states that the rating alone is not consumed and that the correction text is the operative input, since it is the correction — not the rating — that conditions subsequent answers.

One limitation is worth recording. Corrections are global rather than scoped to their author: any correction is eligible to condition any user’s subsequent query. This is defensible for a small internal team and is the reason corrections are inspectable and individually revocable through an administrative interface; it would not be defensible at wider deployment without a review step. That interface earns its place in practice, having surfaced a stored correction of no semantic content that was otherwise eligible to condition prompts indefinitely.

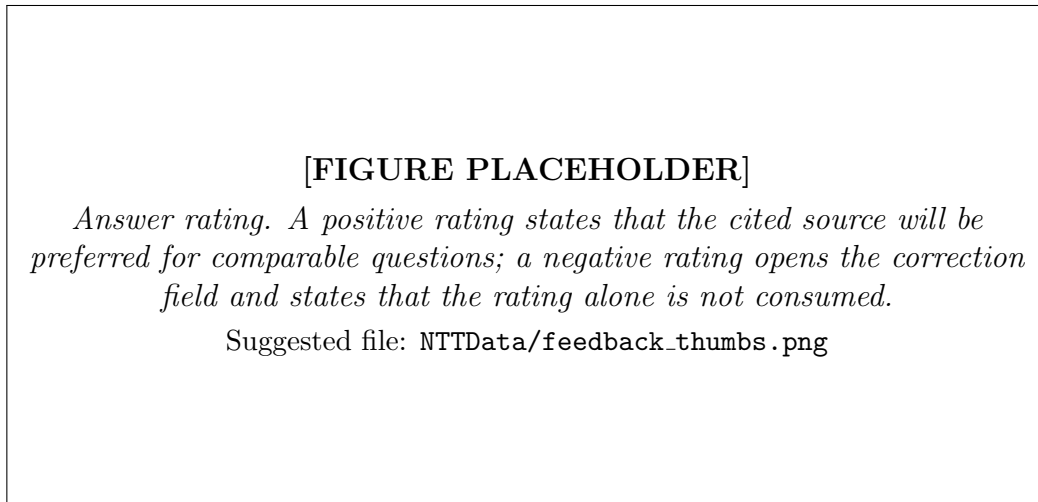


Figure 28: Answer rating. A positive rating states that the cited source will be preferred for comparable questions; a negative rating opens the correction field and states that the rating alone is not consumed.

10 Post-Conditioning: Reconciling Answers with Evidence

Section 9 describes what the generator is asked to do. This section describes what is enforced regardless of whether it complies. The distinction is the operative one under the local-inference constraint of Section 3: a 3B-parameter model follows instructions imperfectly, so every property that must hold is checked after generation rather than requested before it. Each mechanism below addresses a specific failure mode of the generator, and each is verified by the suite of Section 13.

10.1 Artifact Grounding

Resolution steps carry executable artifacts — queries, commands, configuration — and these are the part of an answer an engineer copies. The failure mode this addresses is artifact carry-over between incidents: asked how to remove duplicate customer records, the model returns `python menu.py`, a command documented in an *unrelated* rollback incident, in place of the stored procedure the correct report documents. Retrieval is not implicated — the correct report is selected and its procedure is present in the context — so the check belongs after generation.

The system therefore extracts the identifier tokens of each artifact and requires them to be attested by the retrieved evidence, taken as the full text of the selected reports. An artifact whose identifiers are unattested is removed rather than displayed. The check is contextual rather than a fixed deny-list, which is what makes it usable: the same command `python menu.py` is retained for the rollback incident that documents it and removed from the deduplication answer that does not. Artifacts consisting of a keyword and an ellipsis — `SELECT . . .` — are removed unconditionally, being unrunnable stubs whose presentation as runnable is itself the defect.

10.2 Confidence Calibration

A confidence score emitted by the model is a self-report, and self-reports are unreliable in both directions. Two bounds are therefore imposed from evidence the system holds independently.

A *floor*, because small models under-report when handed a directly matching report: if a selected report scores above the strong-match threshold and the answer contains grounded steps, confidence is raised. *Two ceilings*, because the model over-reports in the two cases where over-reporting is most harmful. An answer containing no resolution steps is capped and marked low-confidence irrespective of its self-report — the reader sees the badge, not the absent list, and an authoritative-looking empty answer is worse than an honest hedge. An answer produced with no selected report at all is capped further still: it derives from the model’s general knowledge, not from anything this estate has done, and must not be presented identically to guidance traced to a real incident. An invented procedure for a subject absent from the corpus is reported at 30% and carries no citation; unbounded, the model self-reports such an answer at 85%.

10.3 Irreversible Operations

Some documented remediations are destructive — truncating a corrupted staging table is a real fix — so suppressing destructive commands would make the assistant useless. The hazard is narrower: a destructive command the model *invented*, presented in the same neutral styling as one a colleague has already run in production — `DROP TABLE production.customers`; offered as step one, uncited and styled like any other step.

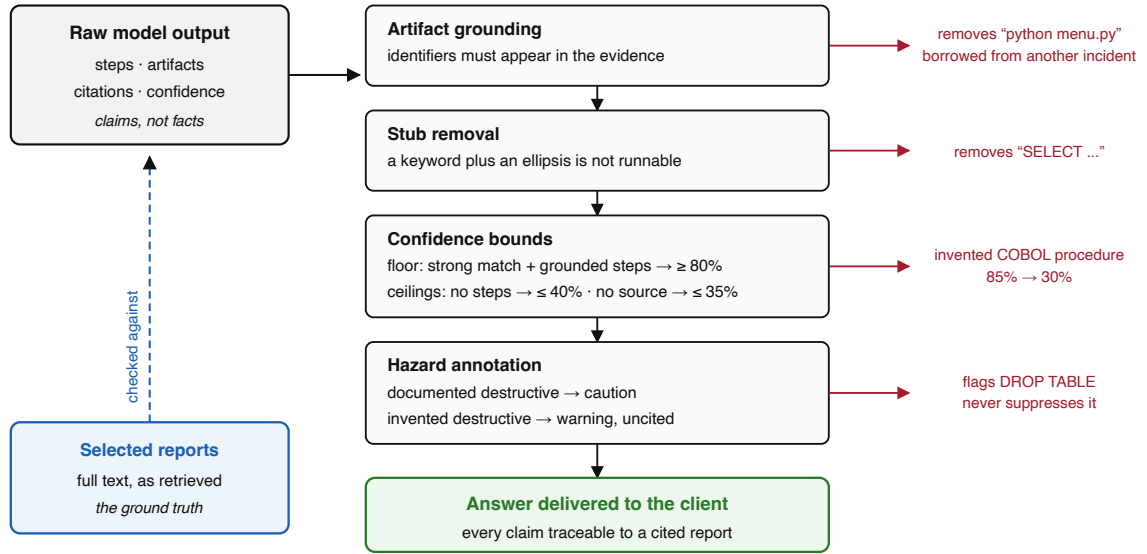
The system detects irreversible operations (unqualified `DROP`, `TRUNCATE`, `DELETE/UPDATE` without a predicate, recursive removal, filesystem creation, infrastructure destruction, unguarded force-push, cache flush, privilege revocation) and annotates the step and the answer. It does not refuse and does not rewrite. What it distinguishes is grounded from ungrounded: a destructive step supported by a report receives a confirmation prompt, while one supported by nothing receives an explicit statement that no report documents

it and that a backup and change record should be verified first. The client renders the former as a caution and the latter as a warning.

Precision was the binding design constraint, since a warning that fires on routine work is one operators learn to dismiss. The patterns distinguish a predicated `DELETE` from an unpredicated one and `--force-with-lease` from a bare `--force`; over a held-out set they flag 10 of 10 destructive commands and 0 of 10 ordinary ones.

10.4 Determinism

The generator decodes near-greedily ($T = 0.1$, fixed seed) rather than at the inference server’s default sampling temperature of 0.8, which is tuned for open-ended dialogue. The task here is the opposite: reproduce a documented procedure faithfully. The cost of sampling at 0.8 is measurable — the same question returns eight grounded steps on one execution and a single vague step on the next, from byte-identical retrieved context — and presents as a quality regression rather than as variance. Under near-greedy decoding repeated executions agree: three consecutive runs of the same query return identical step counts and artifacts. Reproducibility is a quality property in its own right here, since two engineers on the same incident must not receive different procedures — and it is a precondition for the behavioral suite of Section 13, whose failures would otherwise be indistinguishable from resampling.



Each check can only remove content or lower confidence — none can add a claim the evidence does not support.

Figure 29: Post-conditioning. Each check compares the answer against the retrieved evidence and may only remove content or lower confidence; none can add a claim the evidence does not support. The right-hand notes record the observed failure each check was introduced for.

11 Report Synthesis

The corpus is the system’s memory, so its growth determines long-term utility. Because the assistant’s output and the corpus documents share the schema of Section 3, a resolved

dialogue can be projected directly onto a report, inverting the ingestion projection π .

The synthesis maps dialogue elements onto schema fields under a strict grounding rule: the reported symptom is taken from the initiating query, the root cause from the model’s diagnosis, the resolution steps from the generated step sequence, code blocks from the typed artifacts, and cross-references from the cited reports. Fields not determined by the dialogue are left empty rather than filled by inference, and identifying metadata records provenance. Every populated value is therefore traceable to dialogue content.

The synthesized report is validated against the same schema as authored reports and persisted through the same service, so it is indistinguishable downstream: it is editable in the report editor and, once written, becomes retrievable corpus. This closes the loop between the system’s two modules — the assistant consumes the corpus to resolve incidents and extends it with each resolution.

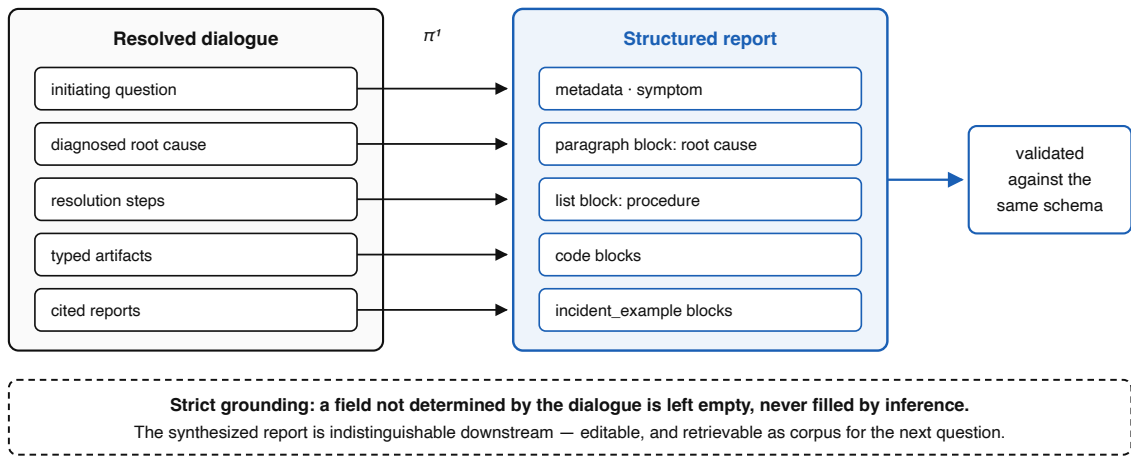


Figure 30: Report synthesis. Each schema field is drawn from a specific dialogue element; fields the dialogue does not determine are left empty rather than inferred, so every populated value stays traceable.

12 Efficiency

Local inference imposes a hardware-bound latency floor (Section 3), so efficiency work cannot target the generation step itself. It targets everything else: the cost paid before generation, and the cost of not repeating generation that has already been performed. Optimization was directed by measurement rather than intuition — profiling the pipeline showed retrieval, selection, and context assembly together accounting for 66 ms, which is negligible beside a generation measured in tens of seconds, and correctly redirected attention to startup and repetition.

12.1 Caching

Two caches address two distinct repetitions.

Embeddings. The corpus is re-encoded on every process start although it rarely changes. Chunk vectors are cached under a digest of the encoder identity and the concatenated chunk texts, so any edit to any report changes the key and invalidates the cache

without an explicit versioning step. Guards cover the failure modes: a shape mismatch or an unreadable file degrades to re-encoding rather than propagating a corrupt index, and writes are atomic so an interrupted write cannot be read as valid. Container start falls from 37 s to 7 s.

Answers. An incident desk repeats questions — a shift asks what the previous shift asked. Shaped answers are cached in a bounded LRU keyed on the *prompt*, not the question. This choice is what makes the cache safe rather than merely fast: the prompt already encodes the retrieved evidence, the dialogue history, and any applicable corrections, so re-indexed reports or a new correction produce a different key and a stale answer cannot be served. It also bounds cross-user exposure without per-user partitioning, since two users share an entry only when their entire input coincided. Weak answers — low-confidence, ungrounded, or abstaining — are deliberately not cached, so a poor result is not pinned for everyone who later asks the same question. A repeated query returns in 0.04 s against 28 s for generation.

12.2 Measured Behavior

Table 6 records the end-to-end figures. Steady-state memory is 658 MB against a 2 GB container bound.

Table 6: End-to-end latency, single user.

Operation	Before	After
Container start to serving	37 s	7 s
Cold start including corpus indexing	—	24 s
Answer, first occurrence	28 s	28 s
Answer, repeated	28 s	0.04 s
Retrieval, selection, context assembly	—	0.066 s

12.3 Concurrency

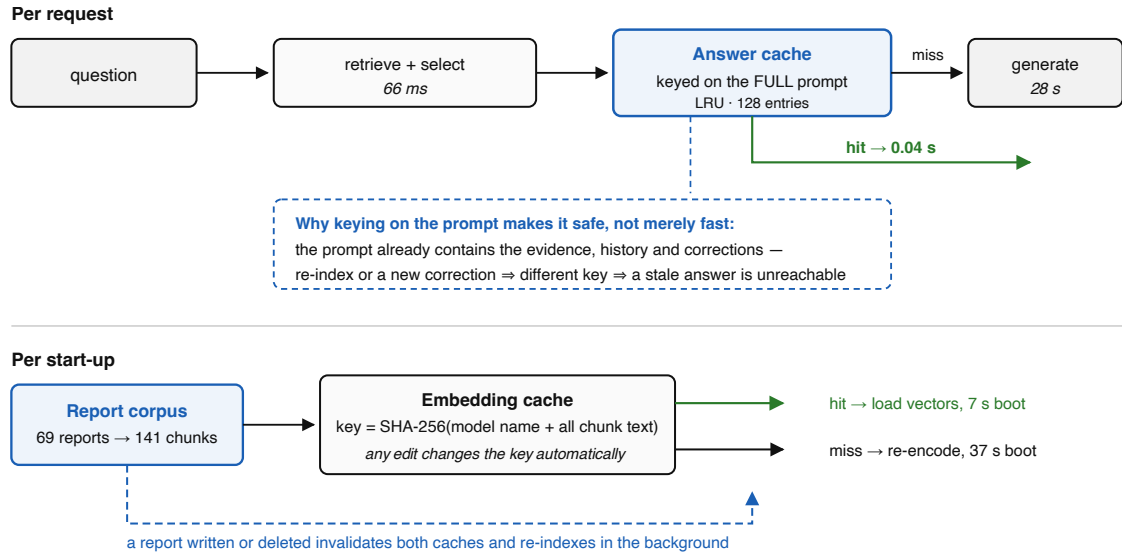


Figure 31: The two caches. The answer cache is keyed on the full prompt, so re-indexing or a new correction yields a different key and a stale answer is unreachable; the embedding cache is keyed on a digest of the corpus text, so any edit invalidates it without an explicit versioning step.

Under concurrent load the inference server serialises generation: four concurrent workers issuing eight distinct queries all succeeded, with no inconsistent answers, but per-request latency ranged from 27s to 133s over 226s of wall time. The request timeout is sized against this queueing behavior rather than against single-request latency; a bound near the 30s single-request figure would fail queued requests spuriously.

The apparent remedy — raising the server’s parallel-slot count — was measured and rejected. With four slots on a single GPU the slots share one accelerator rather than running independently: wall time improved only from 226s to 190s while the *fastest* request degraded from 27s to 62s and resident memory rose from 2.5 GB to 4.8 GB. Serial execution behind the answer cache is the better configuration on this hardware. The measurement is recorded rather than the intuition, since the intuition is wrong.

That experiment also exercised the calibration of Section 10 under adverse conditions: one answer degraded to zero steps under contention, and the confidence ceiling reported it at 40% rather than presenting an empty answer as authoritative — the guard behaving as designed in precisely the circumstances it was written for.

13 Validation Methodology

The retrieval benchmark of Section 8 measures ranking quality. It cannot observe generation behavior, because it does not invoke a generator. This section describes the complementary validation, whose structure follows from a single observation: *every defect that materially affected this system was a generation-behavior defect, and none would have been caught by the retrieval benchmark or by unit tests over stubbed components.*

The cross-incident artifact leak, the unrunnable stub, the mis-calibrated confidence, and the invented destructive command of Section 10 were all first observed by a human running a query by hand. Validation is therefore layered so that each subsequent discovery becomes an automated check rather than an anecdote.

Table 7: Validation layers. Only the last two invoke the deployed model.

Layer	Question it answers	Cases	Cost
Component tests	Do parsing, routing, caching, grounding, and the safety bounds behave as specified?	269	44 s
Corpus sweep	Does <i>every</i> report remain retrievable by a question derived from it?	69	13 s
Behavioral suite	Does the deployed model satisfy the required properties end to end?	16	~5 min
Concurrency probe	Do the shared caches and the index swap hold under simultaneous use?	8–12	~4 min

13.1 Component Tests

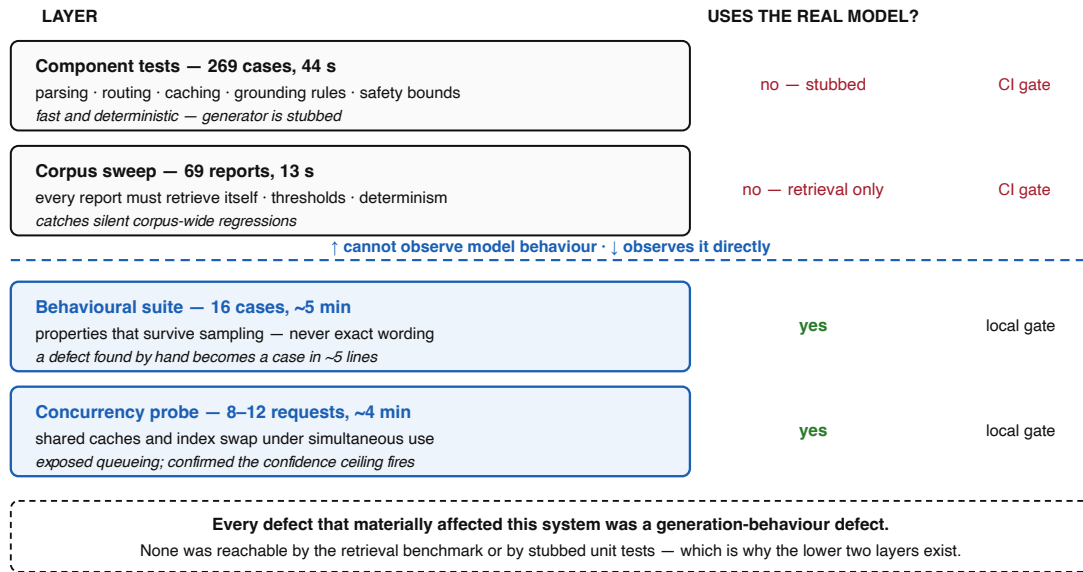


Figure 32: Validation layers. The upper two are fast, deterministic and run in continuous integration, but stub the generator and therefore cannot observe model behavior; the lower two invoke the deployed model and run locally. Every defect that materially affected this system lay below the dividing line.

The component suite substitutes a stub for the generator, which is correct for what it verifies — parsing, intent routing, cache invalidation, grounding rules, confidence bounds, hazard detection — and keeps the suite fast and deterministic. Its structural limitation is precisely that it cannot observe the deployed model, which is why the layers below exist.

13.2 Corpus Sweep

Hand-chosen test cases are chosen by whoever is fixing the defects, so they can pass while the remainder of the corpus regresses unnoticed. The sweep constructs, for each of the 69 indexed reports, a query derived from that report’s own title, and asserts the pipeline selects that report. All 69 pass, with selection scores in $[1.042, 1.044]$ — far above τ_{abs} (Section 7.3).

The sweep runs without a generator and is therefore a continuous-integration gate rather than a manual exercise. It asserts the invariants directly: that each report retrieves itself, that every genuine match clears the absolute threshold, that citation lists stay within the selection cap, that out-of-corpus questions select nothing, and that retrieval is deterministic. It would have detected the duplicate-indexing and title-extraction defects of Section 6 on their own, without waiting for a human to notice a poor answer.

13.3 Behavioral Suite

The behavioral suite drives the deployed system over HTTP with the real model and asserts properties that hold regardless of sampling — which report is cited, that steps are present and actionable, that no artifact is unattested or a stub, that confidence is consistent with content, that conversational and out-of-scope input is not answered with a corpus search. It never asserts exact wording, which would fail on every run for reasons unrelated to correctness. Sixteen cases span conversational routing, adversarial input, under-specified input, five documented incidents, abstention on an uncovered subject, and two safety cases; all pass.

Its economics are what make it worth maintaining: adding a case is a few lines, so a defect found by hand becomes a permanent check in the time it takes to describe it.

13.4 Model Selection as a Measured Decision

The generative model is a configuration parameter, and the constraint of Section 3 shifts leverage away from model scale. Whether that shift actually holds is testable, so both candidate models were run through the identical pipeline over the same sampled reports with the same seed (Table 8).

Table 8: Generation models, identical pipeline and sample. Correctness is a tie.

Model	Reports	Checks	Mean conf.	Mean steps	Median latency
3B (deployed)	15/15	60/60	88.7%	4.0	28.1 s
8B	15/15	60/60	92.0%	3.4	40.3 s

Both models answer every report and satisfy every property check. The larger model reports greater confidence *in itself* while producing *fewer* resolution steps — on two reports it returned a single step where the smaller model returned three and four — at 43% higher latency. The tie in correctness is direct evidence for the design principle stated in Section 1: with schema-aware representation, hybrid retrieval, narrow selection, and post-conditioning in place, the residual contribution of model scale on this corpus is not measurable. The smaller model is retained not as a concession to the hardware but as the better-performing configuration under the measurement.

14 Discussion

14.1 Interpretation

The central result is the per-class decomposition of Table 4. It shows that the hybrid’s aggregate advantage is not a uniform improvement but the union of two disjoint competencies: the dense component contributes on paraphrastic queries where it is already saturated, and the sparse component contributes on identifier queries where the dense component fails almost completely (0.221). The fusion attains the upper envelope precisely because the failure modes are disjoint on this query distribution.

The secondary result is practical. Both the representation analysis (Section 6) and the k -ablation (Section 8.3) show that this system’s quality is governed by *what is retrieved and how it is represented*, not by generative capacity. Under a constraint that caps model scale, this is the lever worth pulling: the projection π , the retrieval method, and the context bound, rather than a larger model.

These results carry two caveats worth stating plainly. The benchmark queries are synthesized from the documents they retrieve, so the reported figures are best read as an upper bound on field performance rather than a field measurement; and the corpus, at under seventy reports, is small enough that the selection threshold τ_{abs} of Section 7.3 and the retrieval behavior at scale would both need re-checking against a substantially larger corpus. Neither caveat changes the comparative conclusion — dense, sparse, and hybrid are measured under the same conditions — but both matter when reading the absolute numbers.

14.2 Conclusion

This project delivers a locally-hosted, retrieval-augmented incident assistant whose retrieval stage fuses dense and sparse indices by Reciprocal Rank Fusion. On a 272-query benchmark, the hybrid attains $\text{Recall@5} = 1.000$ and $\text{MRR} = 0.987$ against 0.768 and 0.678 for dense-only retrieval, with the gain localized by per-class analysis to identifier queries where dense retrieval attains 0.221. Ablations establish invariance to chunk size on this corpus and saturation at $k \geq 3$, which bounds the generation context without recall loss.

Ranking, however, was only the first half of the problem. A retriever always returns its top-ranked candidates, so a pipeline that cites them unconditionally attributes an answer to whichever document ranked first even when none is relevant; and a small model instructed to ground its output does so imperfectly, carrying artifacts between incidents, over-reporting confidence on answers containing nothing actionable, and proposing destructive operations no report documents. The system places a selection stage between retrieval and generation that admits the empty set, and a post-conditioning stage after generation that reconciles the answer against the evidence — removing unattested artifacts, bounding confidence by content, and distinguishing documented destructive procedures from invented ones. These are enforced properties rather than instructions, which is what makes them hold under a model that follows instructions imperfectly.

The measured outcome supports the design decision stated in Section 1. Against a generative model of $2.7\times$ the parameter count, evaluated through an identical pipeline on identical questions, correctness is a tie — both answer all 15 sampled reports and satisfy all 60 property checks — while the larger model is 43% slower and produces *fewer* resolution steps. Under a local-inference constraint, the decisive variable is not

model scale but the quality of the context constructed for it: schema-aware document representation, hybrid retrieval, narrow and thresholded selection, and post-conditioning against the retrieved evidence. The system satisfies its privacy constraint throughout, executing all inference locally.

The platform of Section 4 supports a complementary claim: the properties evaluated above belong to the retrieval and generation stack itself rather than to any particular deployment of it. The stack operates behind authentication and on top of independently-operated storage without modification — the request pipeline, the selection thresholds of Table 2, and the post-conditioning checks of Section 10 depend on that platform only for persistence. The authentication audit is, on its own, a finding worth stating plainly: an endpoint reachable without a credential — report deletion among them — is a defect class invisible to correctness review of individual handlers, since each can be correct against an assumption about where authentication happens, and visible only to a check run against the deployed system as a whole. The discipline that produced the retrieval and generation results — measure the deployed system rather than trust the design — is what surfaces it.

References

- [1] Robertson, S., & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 3(4), 333–389.
- [2] Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. In Proceedings of the 32nd ACM SIGIR Conference on Research and Development in Information Retrieval, 758–759.
- [3] Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. In Proceedings of EMNLP-IJCNLP, 3982–3992.
- [4] Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. In Advances in Neural Information Processing Systems 33, 9459–9474.